

KakaoCloud
Kubernetes Engine
Intermediate Course

T6-03 이론 교재

kakaocloud

KakaoCloud Kubernetes Engine Intermediate Course

본 과정에서는 웹 서비스 (WS), 웹 애플리케이션 서버 (WAS), 데이터베이스(DB)를 컨테이너 형태로 구성하고 쿠버네티스 환경에서 운영하는 방법을 소개합니다. 교육은 쿠버네티스 클러스터 설정, 컨테이너 이미지 빌드, 배포, 모니터링, 보안 등을 다루며, KakaoCloud의 인프라와 서비스를 활용하여 MSA 웹 서비스를 최적화된 방식으로 개발 및 관리하는 데 필요한 핵심 기술과 노하우를 제공합니다.

강의안내

구성	세부내용
OT	강의 안내
Lab.1	Lab1 : 기본 환경 구성 (VPC 생성 , MySQL 인스턴스 그룹 생성, 사용자 액세스 키 생성)
SECTION.1 MSA 등장과 가상화 기술	MSA 등장과 컨테이너 기술
SECTION.02 Kubernetes	Kubernetes
SECTION.03 카카오 클라우드 Container Pack - Kubernetes Engine	카카오 클라우드 Kubernetes Engine
Lab.2	Lab2 : Kubernetes Engine Cluster 생성
Lab.3	Lab3 : Kubernetes Engine Cluster 관리 VM 생성 실습 (1)
	Lab3 : Kubernetes Engine Cluster 관리 VM 생성 실습 (2)
SECTION.04 카카오 클라우드 Container Pack - Container Registry	카카오 클라우드의 Container Registry
Lab.4	Lab4 : Container Image 만들기
SECTION.05 Kubernetes Engine의 클러스터 관리 방법	Kubernetes Engine의 클러스터 관리 방법
SECTION.06 Kubernetes Engine의 사용자 정의 설정 및 웹서버 배포	Kubernetes Engine만의 사용자 정의 설정 및 웹서버 배포
Lab.5	Lab5 : 인그레스 컨트롤러 배포 실습
Lab.6	Lab6 : Kubernetes Engine 클러스터에 웹서버 수동 배포 실습
Lab.7	Lab7 : Kubernetes Engine 활용하기
Lab.8	Lab8 : Kubernetes Engine 클러스터에 웹서버 자동화 배포하기
Lab.9	Lab9 : HPA(Horizontal Pod Autoscaler) 테스트
Lab.10	Lab10 : DNS 서비스를 통해 도메인 주소 연결
Lab.11	Lab11 : 리소스 삭제 / 마무리

CONTENTS

Lab1 : 심화 교육 기본 환경 구성	06
-----------------------	----

01. MSA 등장과 가상화 기술

소프트웨어 개발의 진화 과정	10
모놀리식 아키텍처	11
마이크로 서비스 아키텍처	12
가상화 기술이란 ?	13
가상머신과 컨테이너의 비교	14
MSA의 발전과 가상화 기술	15

02. Kubernetes

쿠버네티스(Kubernetes) 개요	17
쿠버네티스(Kubernetes)의 주요 특징	20
쿠버네티스(Kubernetes)의 다양한 환경 지원	23
쿠버네티스(Kubernetes)의 로드 밸런싱	24
아키텍처와 구성 요소	25

03. 카카오클라우드 Container Pack - Kubernetes Engine

카카오클라우드 Kubernetes Engine이란?	45
Kubernetes Engine 아키텍처	46
Kubernetes Cluster 사용자 접근 방식	47
클러스터 생성 및 관리	48
노드 풀 정보 확인	52
노드 풀 생성 및 관리	55
노드 상세 정보	60
KakaoCloud의 Kubernetes Engine 특징	61
Lab2: Kubernetes Engine Cluster 생성	65
Lab3 : Kubernetes Engine Cluster 관리 VM 생성 실습	66

04. 카카오클라우드 Container Pack - Container Registry

Container Registry란?	68
Container Registry 리포지토리 생성	70
Container Registry 이미지 URI 구조 및 리소스 쿼리	71
Container Registry 이미지 관리	72
Lab4 : Container Image 만들기	74

05. Kubernetes Engine의 클러스터 관리 방법

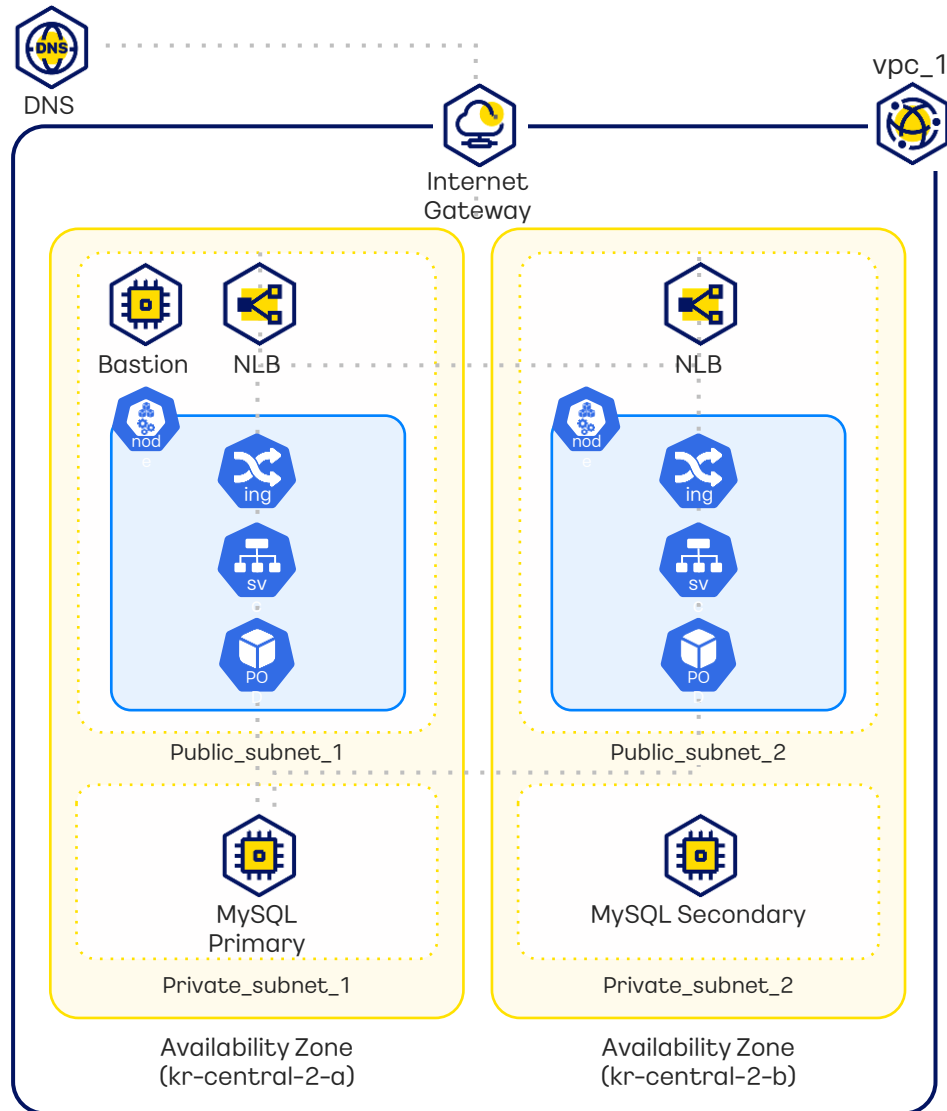
Console UI를 통한 관리	76
HTTP와 gRPC의 사용	77
Kubecti 이란?	78
IaC를 통한 관리	79
Helm Chart	80
Kubernetes Engine의 IAM 역할 관리	81
Kubernetes Engine의 지원 노드 이미지	82

06. Kubernetes Engine의 사용자 정의 설정 및 웹서버 배포

인그레스 컨트롤러 배포	84
Load Balancer 생성 및 삭제	85
클러스터에서 영구 볼륨을 사용하기 위한 방법	86
Kubernetes Engine에서 제공하는 Provisioner	87
루트 볼륨 파티션 테이블 형식 변경	88
Lab5 : 인그레스 컨트롤러 배포	89
Lab6 : Kubernetes Engine 클러스터에 웹서버 수동 배포	90
Lab7 : Kubernetes Engine 활용하기	91
Lab8 : Kubernetes Engine 클러스터에 웹서버 자동화 배포	92
Lab9 : HPA (Horizontal Pod Autoscaler) 테스트	93
Lab10 : DNS 서비스를 통해 도메인 주소 연결	94
마무리: 리소스 삭제하기	95

PREVIEW OF LAB

실습을 통해 카카오클라우드에 구축할 아키텍처는 아래와 같습니다.



Lab1 : 기본 환경 구성

SECTION.01

(실습 교재 4 page, [Github Link](#))

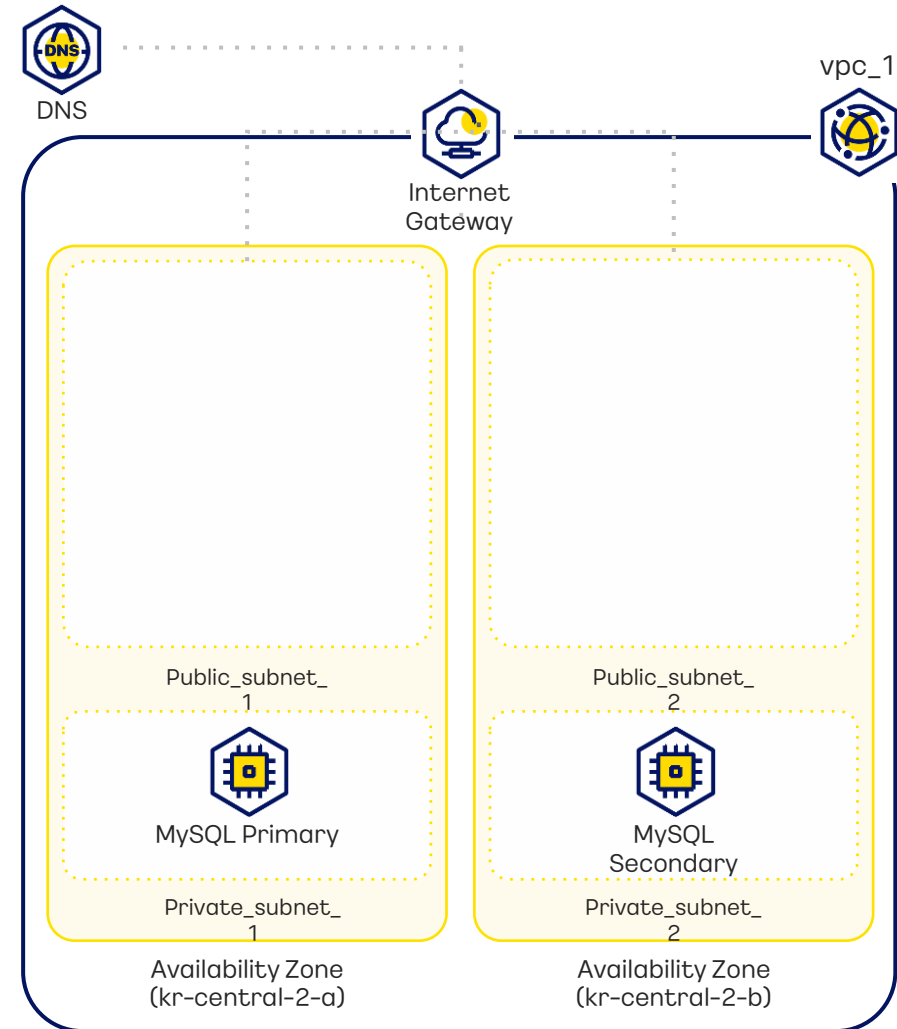


Lab

클러스터 구축을 위한 프로젝트 기본 환경 실습을 진행합니다. VPC, Subnet, MySQL을 생성합니다.

목차

- 1 VPC 생성 (Demo)
- 2 MySQL 인스턴스 그룹 생성
- 3 사용자 액세스 키 생성

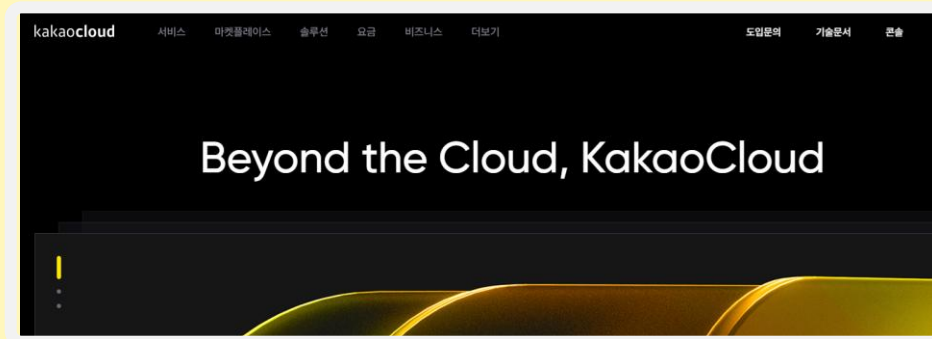


KakaoCloud
Kubernetes Engine
Intermediate Course

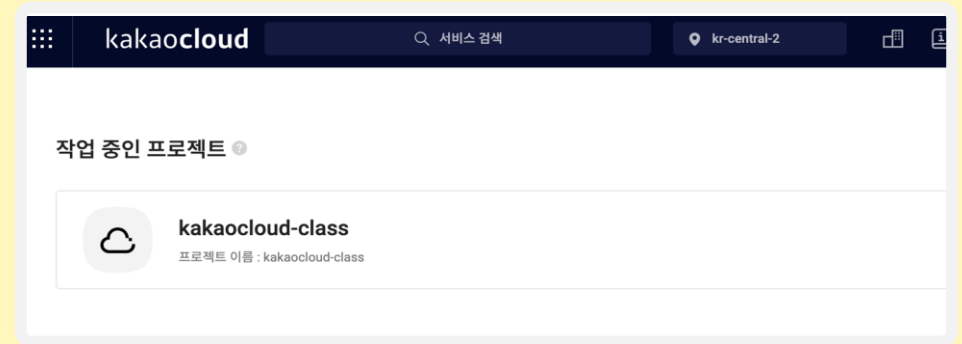
kakaocloud

카카오클라우드 관련 사이트

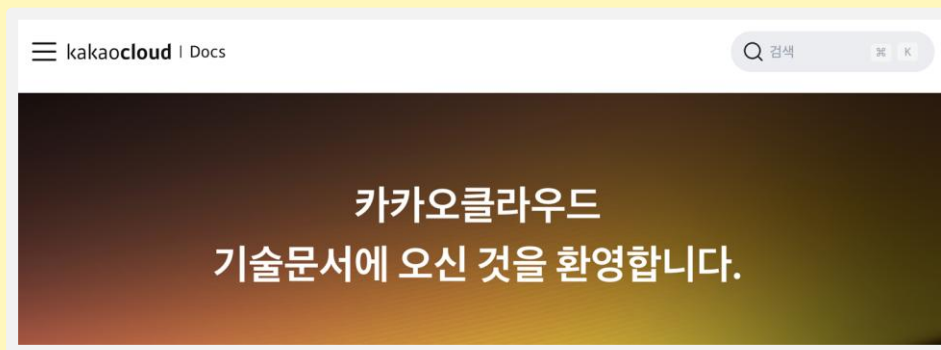
1 카카오클라우드
<https://kakaocloud.com/>



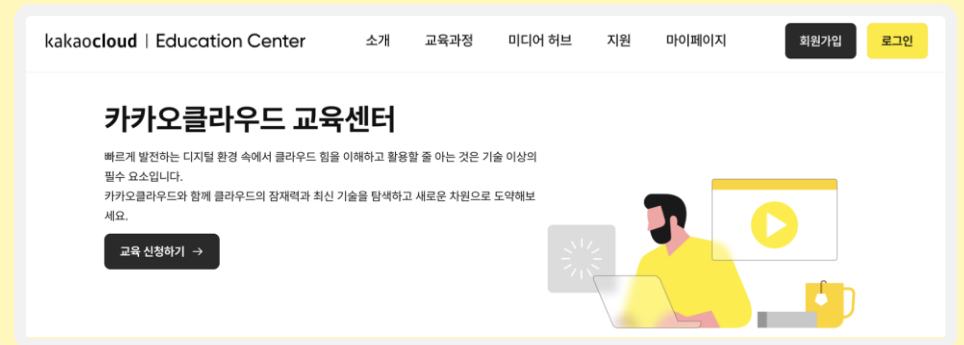
2 카카오클라우드 콘솔
<https://console.kakaocloud.com>



3 카카오클라우드 기술문서
<https://docs.kakaocloud.com>



4 카카오클라우드 교육센터
<https://edu.kakaocloud.com>



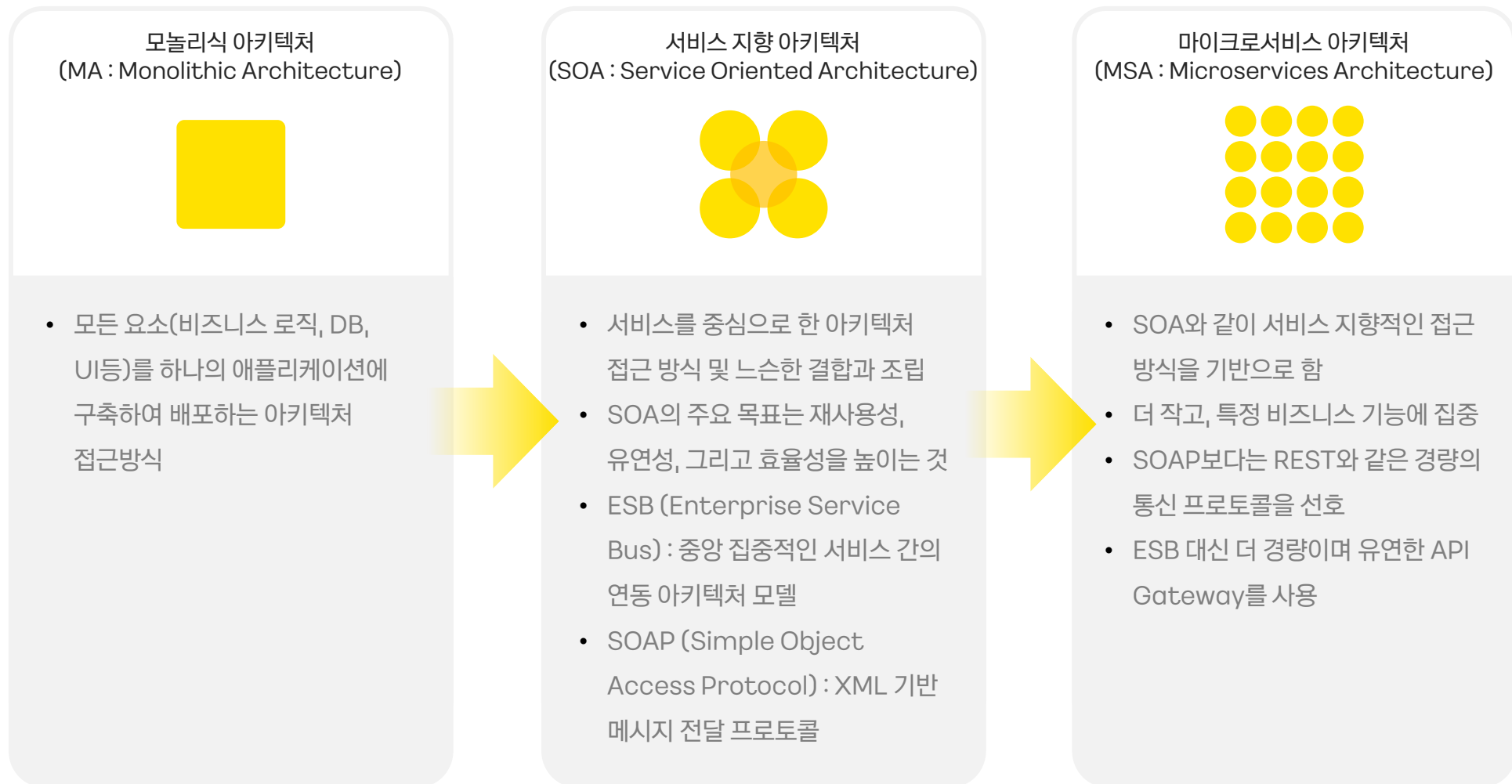
MSA 등장과 가상화 기술

SECTION.01

MSA 등장과 가상화 기술

SECTION.01

소프트웨어 개발의 진화 과정



MSA 등장과 가상화 기술

SECTION.01

모놀리식 아키텍처

✓ 모놀리식 아키텍처(MA : Monolithic Architecture)

- S/W 애플리케이션을 하나의 단일 코드베이스로 구성
- 애플리케이션의 모든 기능과 컴포넌트가 단일한 실행 파일 또는 코드베이스에 통합되어 있음

장점

- 소규모 프로젝트일 경우 형태가 단순하며 유지보수가 편리
- 빠르게 간단한 서비스를 만들 수 있음

단점

- 부분 장애가 전체 서비스의 장애로 확대될 수 있음
- 전체 시스템 구조 파악이 어려움
- 서비스의 스케일아웃(Scale-out)이 어려움
- 서비스 변경이 어렵고, 수정 사항 파악이 힘들
- 빌드 시간 및 테스트, 배포 시간의 급증

MSA 등장과 가상화 기술

SECTION.01

마이크로 서비스 아키텍처

✓ 마이크로 서비스 아키텍처(MSA : Microservices Architecture)

- 마이크로 서비스 아키텍처(MSA)란? : 모놀리식 아키텍처의 한계를 극복하기 위해 각 서비스를 독립적으로 개발, 배포, 운영할 수 있는 아키텍처
- 특히 빠른 개발과 배포의 요구에 부합하여 현대적인 S/W 개발에 적합한 방식

장점

- 빠른 배포
 - 각 서비스를 개별로 나누어 배포 가능
 - 빠른 배포 가능
- 확장성
 - 특정 서비스에 대한 확장성(scale-out) 유리
 - 클라우드 기반 서비스 사용에 적합
- 장애 대응
 - 일부 장애가 전체 서비스로 확장될 가능성 적음
 - 부분적 발생 장애 대한 격리 수월

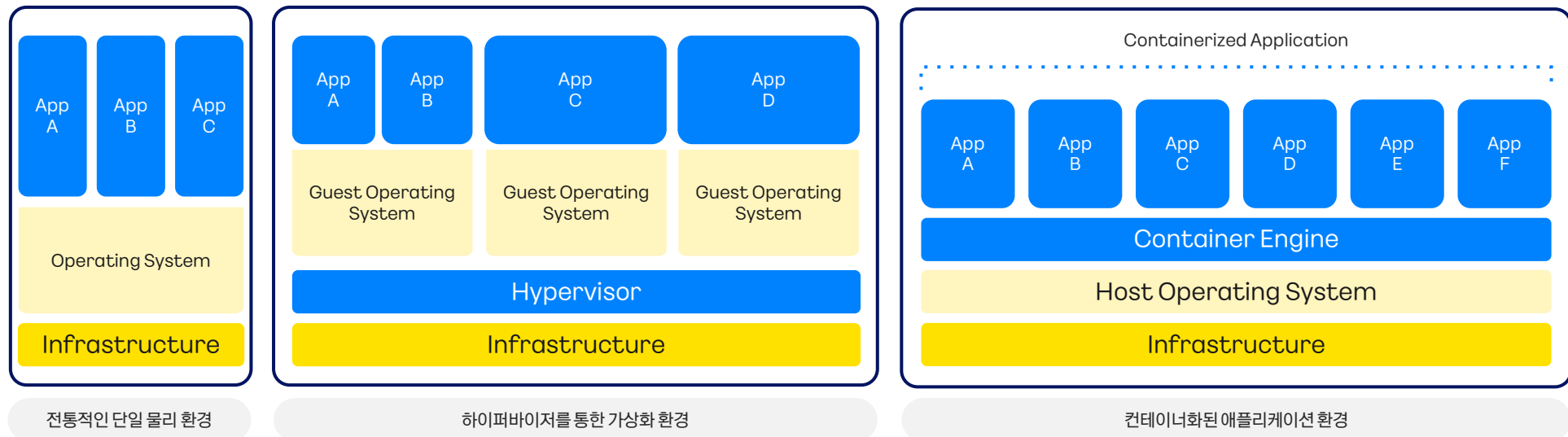
단점

- 서비스가 모두 분산되어 있기 때문에 MSA는 모놀리식에 비해 상대적으로 복잡

MSA 등장과 가상화 기술

MSA는 서비스의 독립적인 자원 사용(자원 격리), 독립적 배포, 유연한 확장성이 필요

- ✓ **가상화 기술이란 ?**
 - H/W, S/W를 사용하여 실제 리소스를 추상화하고, 여러 개의 가상 환경을 생성하는 기술
- ✓ **가상 머신(Virtual Machine)**
 - 하나의 물리적 서버에서 여러 개의 독립적인 가상 서버를 운영할 수 있도록 하는 기술
- ✓ **컨테이너 가상화(Containerization)**
 - 응용 프로그램 및 모든 종속성을 패키지화하여 이식성을 향상시키고 빠른 배포 및 확장이 가능한 기술



MSA 등장과 가상화 기술

SECTION.01

가상머신과 컨테이너의 비교

특징	가상머신(Virtual Machine)	컨테이너(Container)
작동 방식	- 하드웨어 수준의 가상화 - 하이퍼바이저를 통한 게스트 OS 실행	- OS 수준의 가상화 - 컨테이너 엔진을 통한 실행
핵심 기술	- 하이퍼바이저(예 : VMware, Hyper-V, KVM) - 각 VM에 별도의 OS 존재	- 컨테이너 엔진(예 : Docker) - OS의 커널 공유
개발 제어 권한	- OS 레벨에서 완전한 제어 - 애플리케이션과 환경 설정 가능	- 애플리케이션 레벨의 설정 - OS 설정에는 제한
확장성	- 확장성 보통 - 가상 하드웨어 자원에 기반한 확장/축소 가능	- 확장성 높음 - 마이크로서비스 아키텍처 기반으로 한 Scale-out으로 쉽게 확장/축소 가능
크기	크기 큼(GB 단위), 전체 게스트 OS와 애플리케이션을 포함	크기 작음(MB 단위), 필요한 애플리케이션 종속성만 포함
이식성	- 이식성 낮음 - 가상화된 환경 간 이동은 가능하지만 더 복잡할 수 있음	- 이식성 매우 높음 - 다양한 환경(온프레미스, 클라우드)으로 쉽게 이동 가능
시작 시간	시작 시간 느림, 분 단위로 시작할 수 있음	시작시간 빠름, 초 단위로 시작 가능
리소스 효율성	- 리소스 효율성 낮음 - 각 VM이 별도의 OS를 가지므로 더 많은 리소스를 차지함	- 리소스 효율성 높음 - OS 커널을 공유하므로 오버헤드가 적음
보안	격리 수준이 높음, 완전히 분리된 환경	격리 수준이 낮음, 커널을 공유함

MSA 등장과 가상화 기술

SECTION.01

MSA 발전과 가상화 기술

MSA 발전과 가상화 기술의 연관성

✓ 독립적 배포 및 확장성

- 마이크로 서비스는 각각이 독립적으로 배포 가능하며, 필요에 따라 개별적으로 확장할 수 있는 환경이 필요
- 컨테이너 기술은 각 서비스를 격리 가능한 환경을 제공

✓ 스케일링 용이성

- 마이크로 서비스는 서비스 간 독립적인 배포와 확장이 필요해 스케일링이 중요
- 컨테이너 기술은 scale-out을 빠르고 저렴하게 스케일링이 가능

✓ 배포 및 유지보수 용이성

- 마이크로 서비스 특성 상 배포와 롤백 등 버전관리가 잦음
- 컨테이너 이미지나 가상머신 스냅샷을 사용해 개발환경에서 운영환경으로의 일관성 있는 배포가 가능하며, 빠른 배포 속도를 보장

✓ 환경 및 장애 격리

- 마이크로 서비스 아키텍처에서 하나의 서비스에서 발생한 장애가 다른 서비스로 영향 미치는 것을 방지하는 것이 중요
- 컨테이너 기술의 격리된 환경, 컨테이너 오케스트레이션, 셀프 힐링 등이 마이크로 서비스 아키텍처에서의 장애 격리를 강화하고 안전성을 보장
- 가상머신 기술을 통해 확보된 리소스(CPU, 메모리, 스토리지)를 상황에 맞게 컨테이너 클러스터 환경에 분배하여 잠재적인 보안 위협으로부터 보호하고 예측 가능한 성능을 제공

Kubernetes

SECTION.02

Kubernetes

SECTION.02

쿠버네티스(Kubernetes) 개요

쿠버네티스의 등장 배경



소프트웨어 개발의 변화: 모놀리식 → MSA

- 모놀리식 아키텍처의 한계가 드러나며 MSA로의 변화가 시작됨



컨테이너화 기술의 발전

- 애플리케이션과 그 종속성을 패키징 및 실행하는 경량화된 방식을 제공하는 컨테이너화 기술 등장
- Docker와 같은 컨테이너 플랫폼 확산
- 애플리케이션을 환경에 비종속적으로 개발 가능한 환경 마련



대규모 컨테이너 관리의 필요성

- 서비스의 규모가 커지며 마이크로 서비스 아키텍처로의 변화로 많은 수의 컨테이너 관리가 요구됨

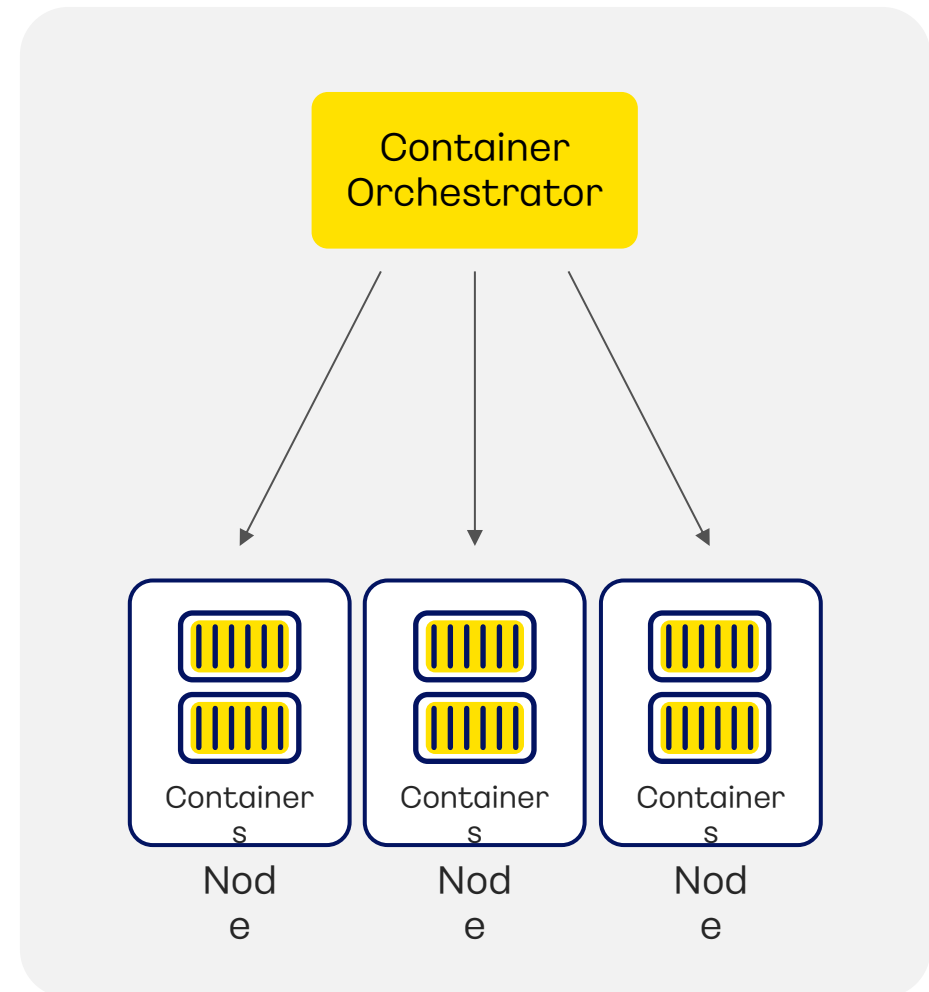
Kubernetes

SECTION.02

쿠버네티스(Kubernetes) 개요

컨테이너 오케스트레이션이란?

여러 개의 컨테이너를 자동으로 배치, 관리, 확장하고, 이들 간의 통신과 서비스 디스커버리를 관리하는 프로세스



Kubernetes

SECTION.02

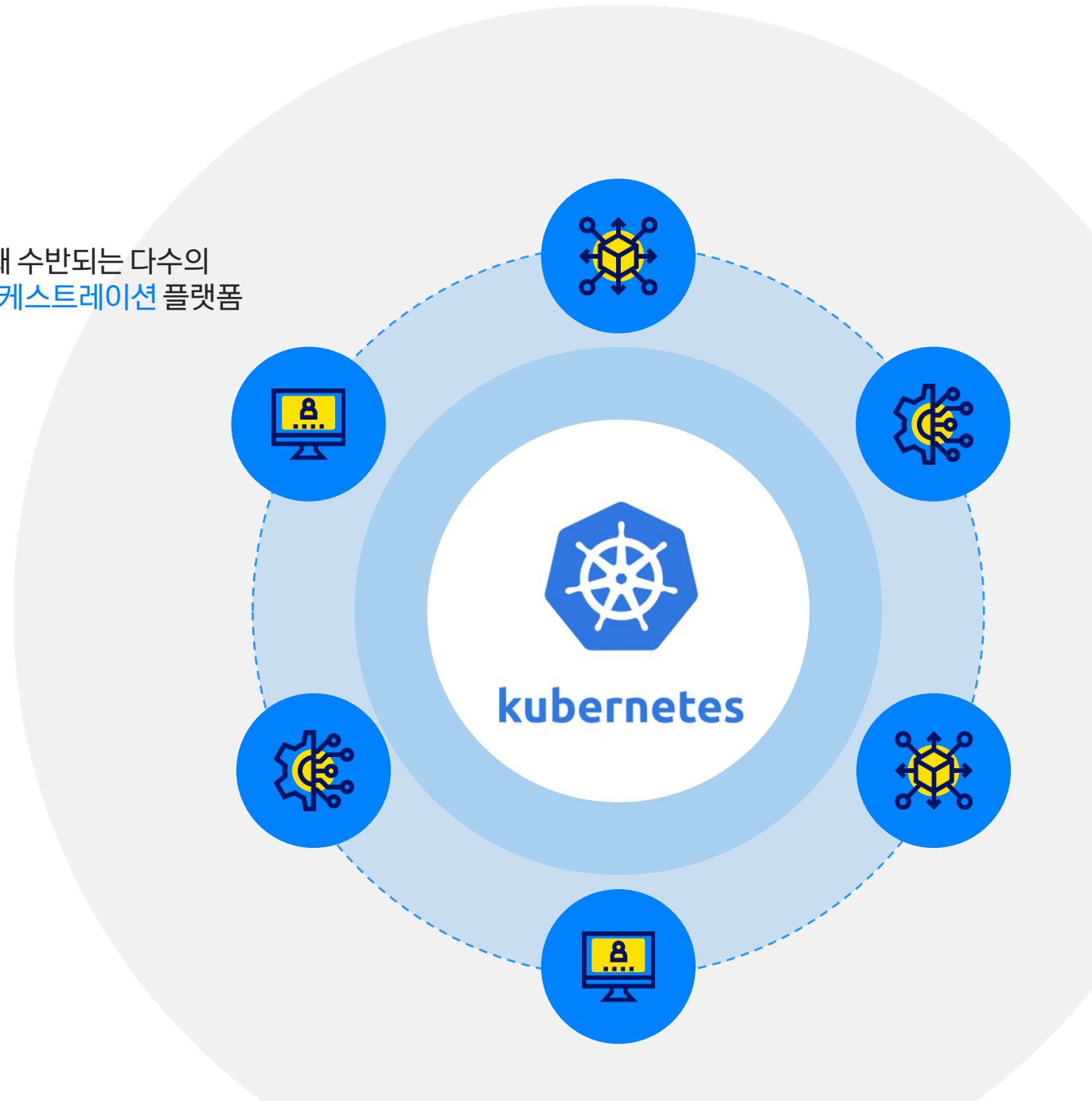
쿠버네티스(Kubernetes) 개요

쿠버네티스(Kubernetes, k8s)란?

컨테이너화된 애플리케이션을 배포, 관리, 확장할 때 수반되는 다수의
수동 프로세스를 자동화하는 오픈소스 **컨테이너 오케스트레이션** 플랫폼

특징

- 선언적 배포
- 자동화된 스케일링
- 자가 치유
- 뛰어난 범용성과 확장성
- 로드밸런싱 지원



Kubernetes

SECTION.02

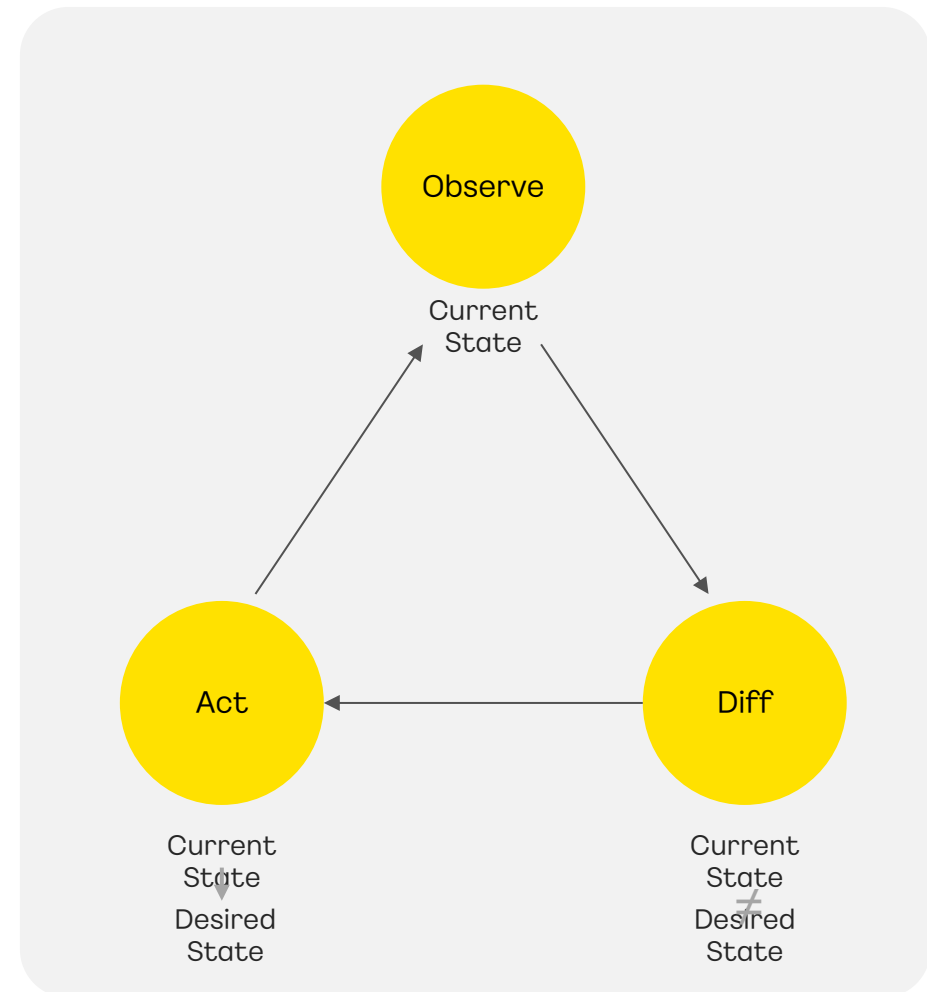
쿠버네티스(Kubernetes)의 주요 특징

선언적 배포(Declarative Deployment)

Imperative (명령적) VS Declarative (선언적)

개발자는 애플리케이션의 상태를 선언적으로 정의
→ 원하는 상태(Desired State)

- 쿠버네티스는 현재 상태(Current State)를 관찰하고 원하는 상태(Desired State)로 수렴시킴
 - 자동화된 스케일링(Automatic Scaling)
 - 자가 치유(Self-Healing) 기능들을 지원



Kubernetes

SECTION.02

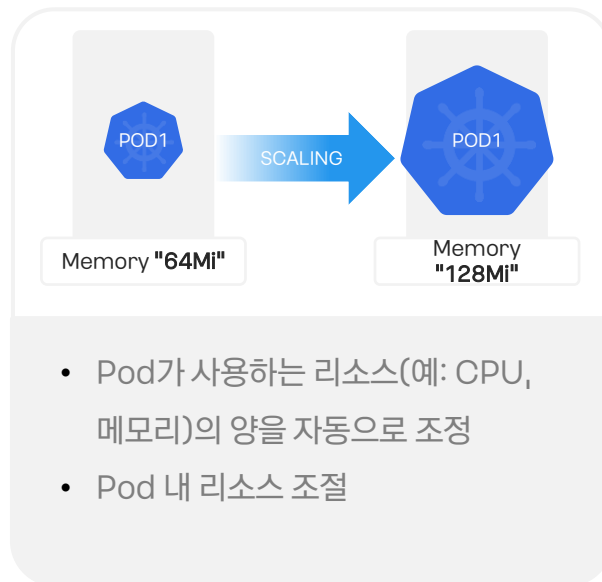
쿠버네티스(Kubernetes)의 주요 특징

자동화된 스케일링 (Automatic Scaling)

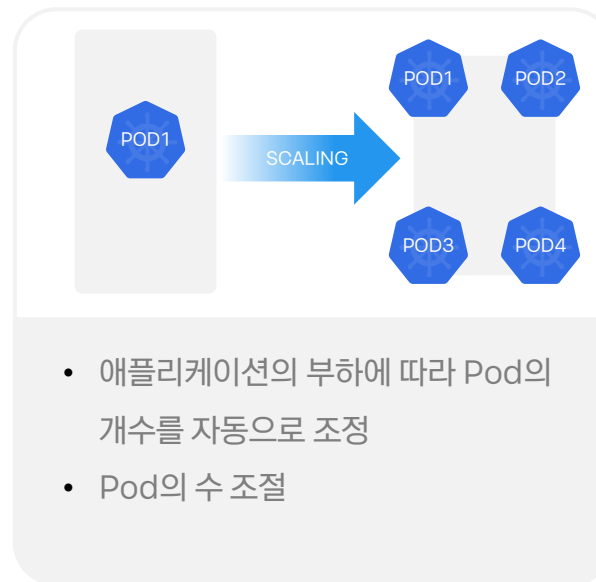
다양한 자동화된 스케일링 메커니즘을 제공하여 애플리케이션의 부하에 따라 자동으로 리소스를 확장하거나 축소 가능
스케일링 하는 타겟에 따라 세 가지 방식으로 정의

- 애플리케이션은 '파드(Pods)'라고 불리는 기본 단위로 실행됨. 각 파드는 하나 이상의 컨테이너로 구성될 수 있음

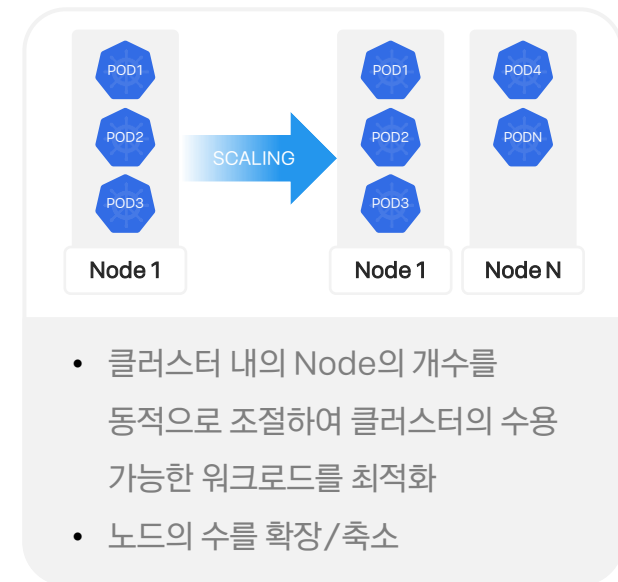
Vertical Pod Autoscaler - VPA



Horizontal Pod Autoscaler - HPA



Cluster Autoscaler - CA



Kubernetes

SECTION.02

쿠버네티스(Kubernetes)의 주요 특징

자가 치유 (Self-Healing)

쿠버네티스는 자가 치유 메커니즘을 통해 실패한 컨테이너를 재시작하고, 노드가 죽는 경우 컨테이너들을 교체하기 위해 다시 스케줄링을 하여 응답하지 않는 컨테이너를 종료시켜 클러스터의 안정성과 가용성을 유지함

ReplicaSets

- ReplicaSets: 특정 파드의 복제본(레플리카)을 정의된 수량만큼 유지 관리하는 쿠버네티스 리소스
- 정의된 Pod의 개수를 관리하고, 만약 노드나 Pod의 장애로 인해 일부 Pod이 사라지거나 종료된 경우에도 정의된 개수를 항상 유지하도록 보장함

Liveness Probes와 Readiness Probes

- 애플리케이션의 건강 상태를 체크
- Liveness Probes :
Pod 내의 애플리케이션이 정상적으로 동작 중인지 확인하고, 문제가 발생하면 해당 Pod 내의 container를 재시작함
- Readiness Probes :
애플리케이션이 트래픽을 수용할 준비가 되었는지 확인하고, 준비되지 않은 Pod는 서비스(Service)로 노출되지 않도록 함

Kubernetes

SECTION.02

쿠버네티스(Kubernetes)의 다양한 환경 지원

범용성과 확장성을 갖춘 컨테이너 오케스트레이션 플랫폼

✓ 클라우드 환경

쿠버네티스는 대부분의 주요 클라우드 (KakaoCloud, AWS, Azure, Google Cloud 등) 플랫폼에서 지원되며, 각 클라우드 플랫폼에서 제공되는 서비스로 손쉽게 배포될 수 있음

✓ 온프레미스 환경

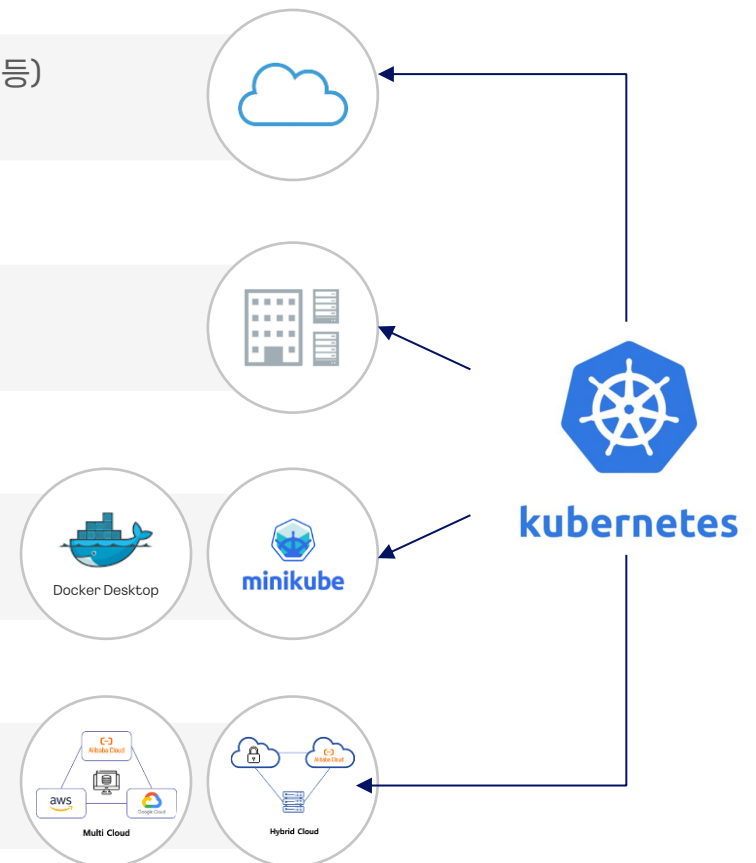
온프레미스 환경에서도 쿠버네티스를 구축하여 관리할 수 있으며, 기존 Infrastructure의 통합을 통해 Hybrid 클라우드 환경을 구성하는 데 도움이 됨

✓ 로컬 개발 환경

쿠버네티스는 로컬 환경(Minikube, Docker Desktop 등)에서도 간단한 클러스터를 설정하고 개발 및 테스트를 위한 미니 클러스터를 생성하는 데 사용됨

✓ 멀티 클라우드 및 하이브리드 클라우드

여러 클라우드 환경에서 쿠버네티스를 통합하여 단일 관리 환경을 제공함 (GCP의 Anthos, Azure Arc 등)



Kubernetes

SECTION.02

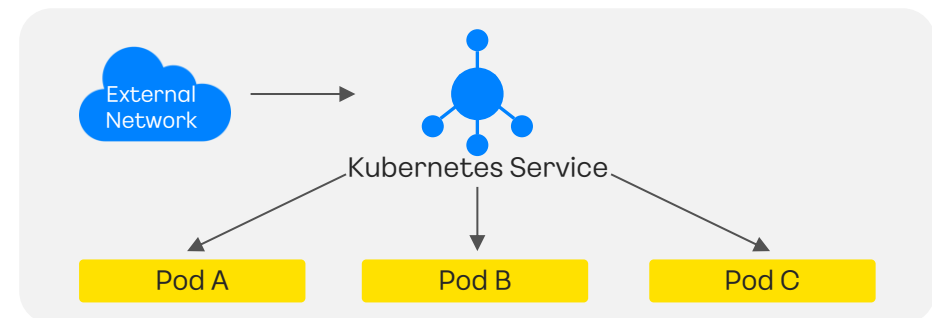
쿠버네티스(Kubernetes)의 로드 밸런싱

로드밸런싱

쿠버네티스에서는 크게 두 가지 방식을 통해 로드 밸런싱을 설정

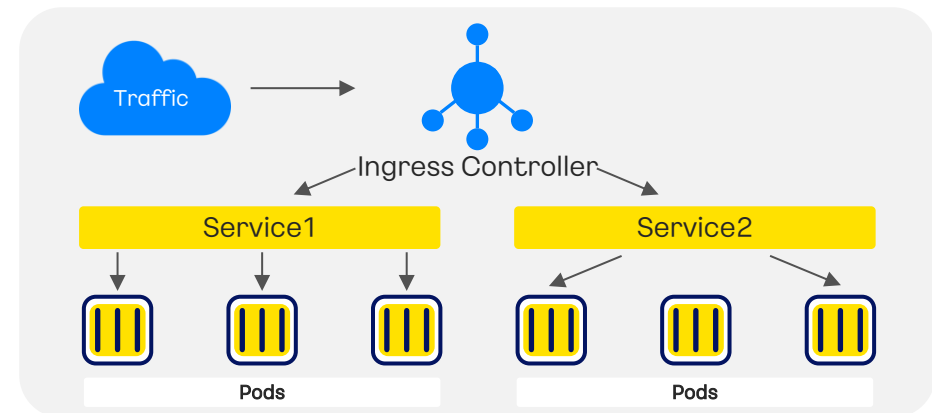
Service 로드밸런싱

- Service 리소스는 쿠버네티스 클러스터 내에서 파드 그룹에 대한 단일 Endpoint를 제공
- 서비스를 통해 내부 또는 외부 트래픽을 해당 파드 그룹으로 로드 밸런싱할 수 있음.



Ingress 로드밸런싱

- Ingress는 HTTP/HTTPS 트래픽을 클러스터 내의 서비스로 라우팅하는 규칙을 정의
- Ingress 컨트롤러를 사용하여 이러한 규칙을 처리하며, 단일 IP 주소를 통해 여러 서비스에 접근할 수 있게 함.
- Ingress를 사용하면 URL 경로, 호스트 이름 등에 따라 트래픽을 다양한 서비스로 분산시킬 수 있음.



Kubernetes

SECTION.02

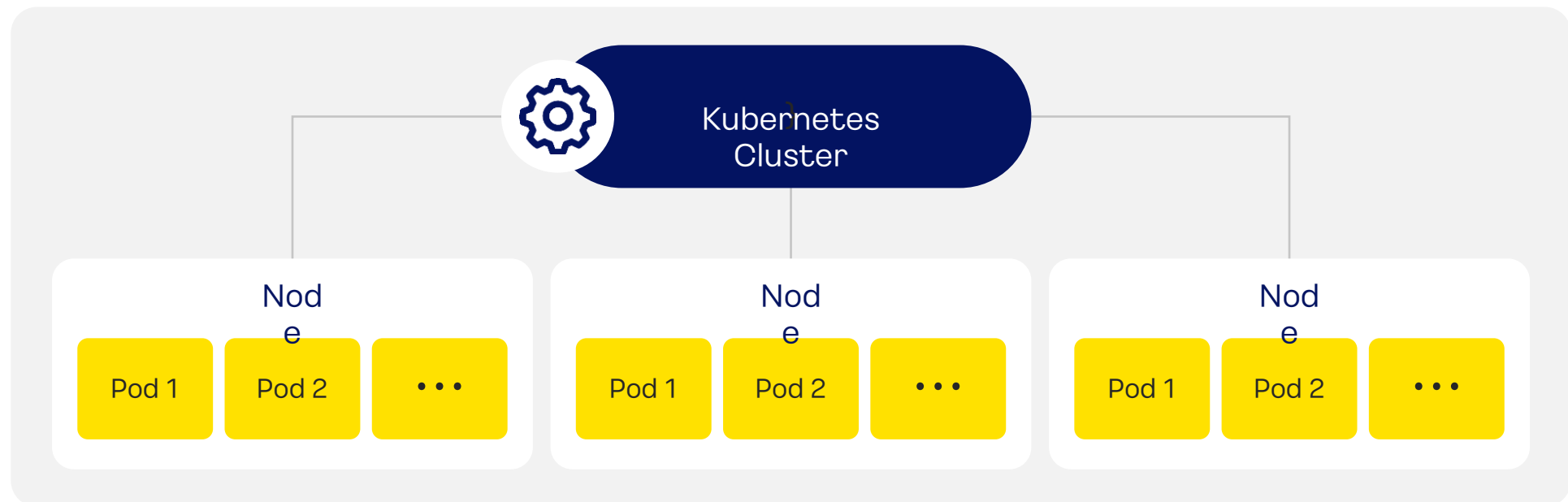
아키텍처와 구성 요소

클러스터란?

쿠버네티스에 의해 관리되는 여러 **노드(Node)**들로 구성된 하나의 컴퓨팅 리소스의 집합

노드란?

쿠버네티스의 노드(Node)는 클러스터 내에서 컨테이너화된 애플리케이션을 실행하는 물리적인 또는 가상의 서버를 나타냄
쿠버네티스 클러스터는 크게 두 가지 유형(**마스터 노드**와 **워커 노드**)의 노드로 구성됨



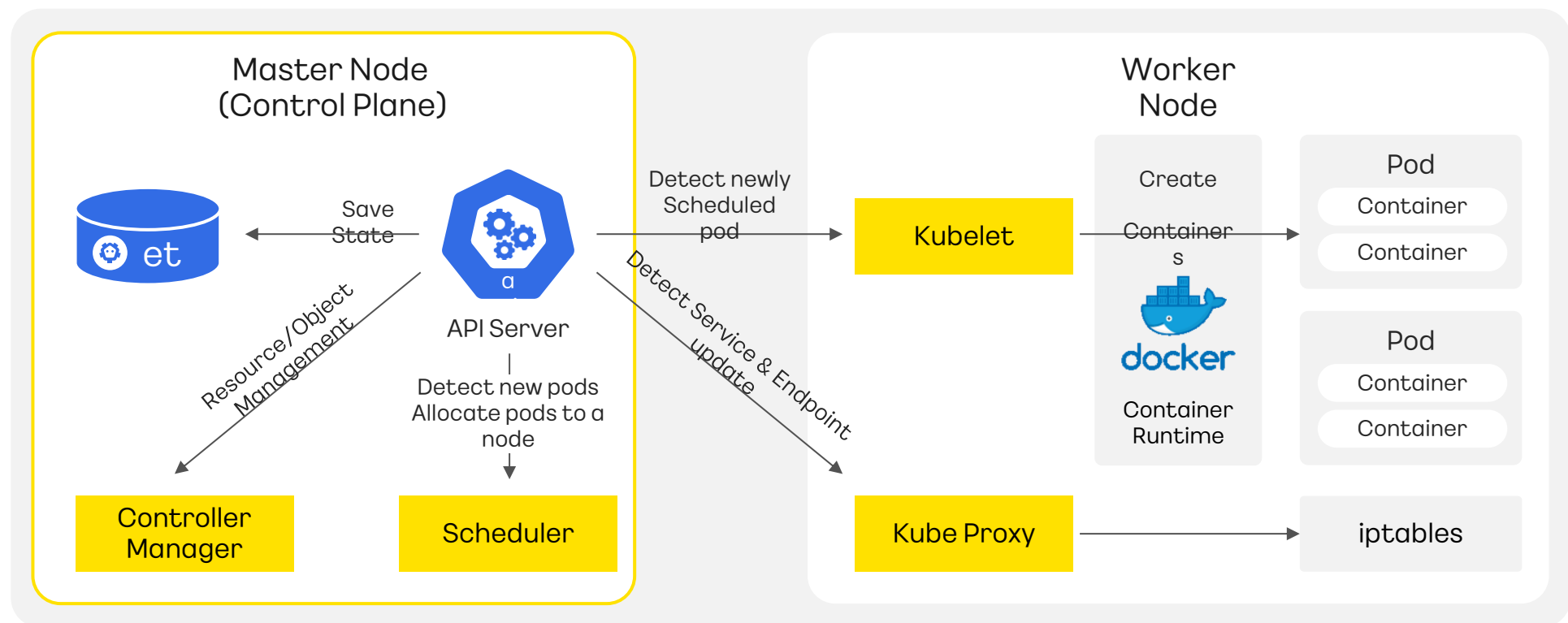
Kubernetes

SECTION.02

아키텍처와 구성 요소

마스터 노드란?

- 클러스터를 제어하고 관리하는 중앙 제어 플레인의 핵심 구성요소
- 전체 클러스터의 상태를 감시하며, 클러스터 내의 노드에게 작업을 할당하고 조절함



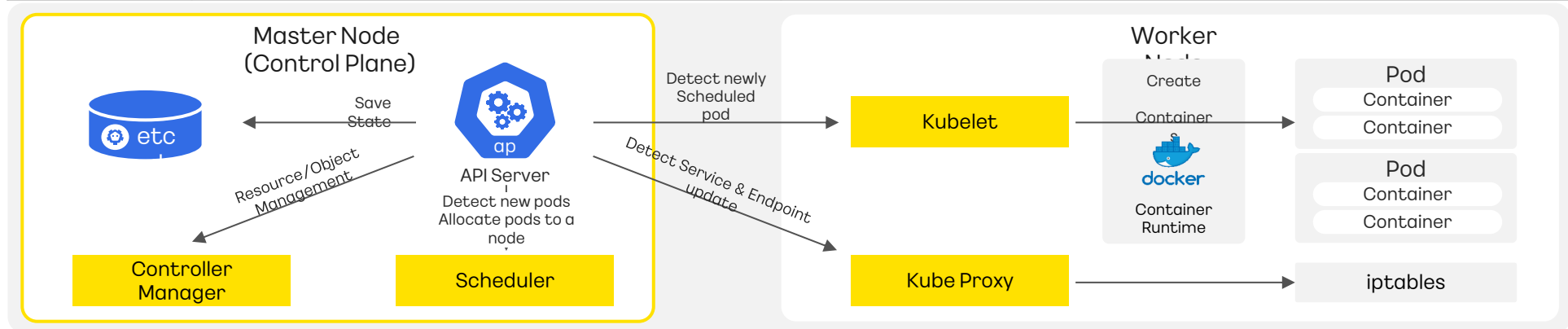
Kubernetes

SECTION.02

아키텍처와 구성 요소

마스터 노드의 구성 요소와 역할

구분	역할	기능
kube-api server	클러스터의 API 서버로, 모든 컴포넌트 및 사용자와의 상호 작용을 담당	- API 엔드포인트를 제공하고, 클러스터 상태에 대한 정보를 노출 - 사용자 및 컴포넌트의 요청을 처리하고, 쿠버네티스 객체의 생성 및 관리를 수행
etcd	쿠버네티스 클러스터의 모든 데이터를 저장하는 분산 키-값 스토어	- 클러스터의 구성, 상태, 메타데이터 등을 저장 - 고가용성을 위해 일반적으로 멀티 인스턴스로 구성
kube-scheduler	새로운 Pod가 어떤 노드에 할당될지 결정하는 스케줄러	- 각 노드의 상태를 고려하여 Pod를 적절한 노드에 할당 - Pod를 생성할 때의 정책에 따라 어떤 노드에 할당할지 결정
kube-controller-manager	다양한 컨트롤러를 실행하여 클러스터의 상태를 원하는 상태로 유지	- Node Controller, ReplicaSets 등의 컨트롤러를 실행하여 클러스터 상태를 유지 - 컨트롤러의 정의에 따라 자동으로 클러스터를 조정하고 관리

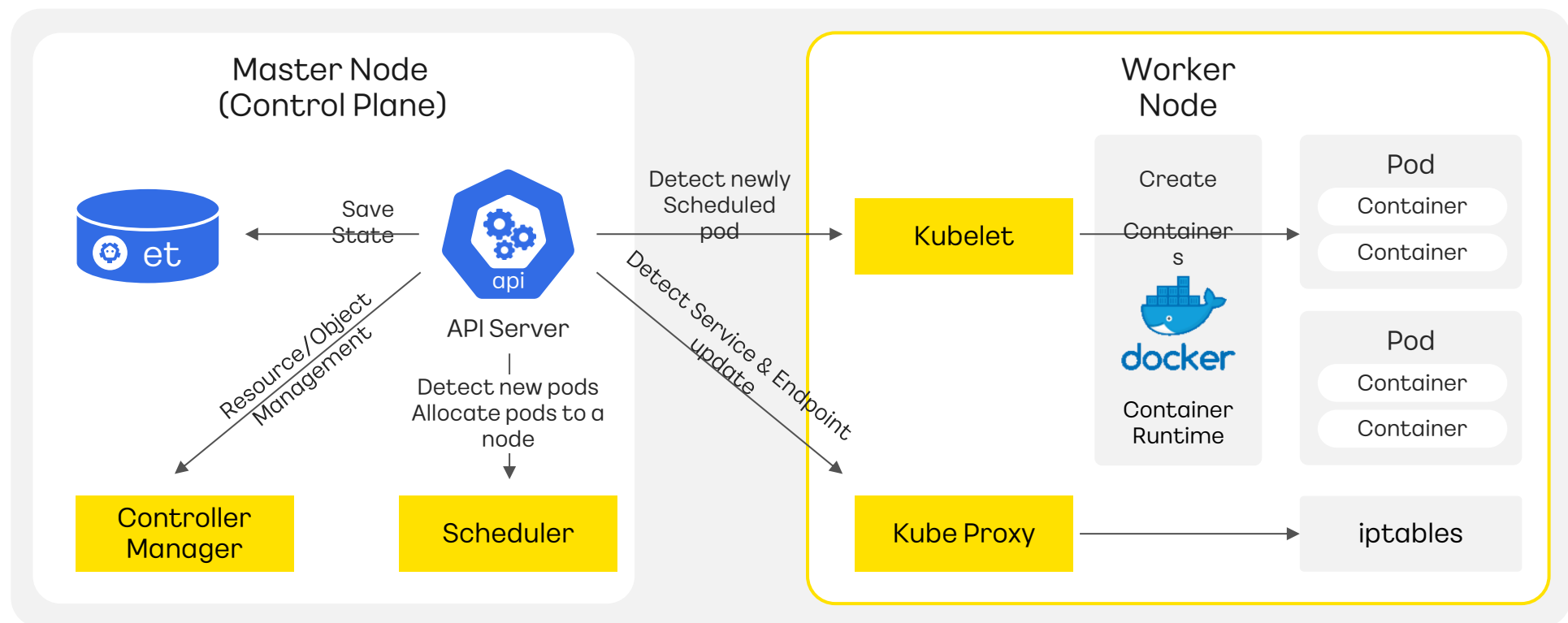


Kubernetes

아키텍처와 구성 요소

워커 노드란?

- 애플리케이션 컨테이너가 실행되는 노드로, 마스터 노드의 지시에 따라 컨테이너를 실행하고 관리함
- 워커 노드는 클러스터에서 실제로 작업을 수행하며, 애플리케이션 컨테이너가 실행되는 환경을 제공함

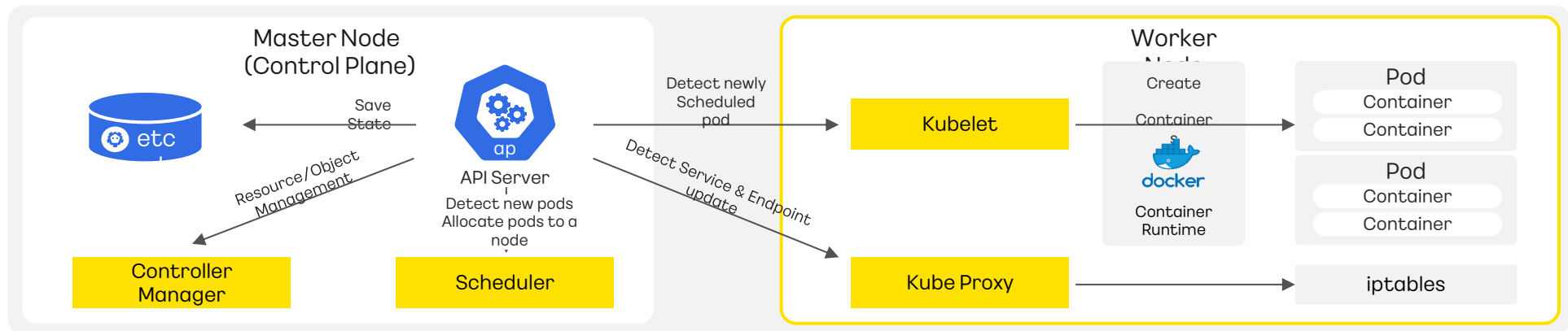


Kubernetes

아키텍처와 구성 요소

워커 노드의 구성 요소와 역할

구분	역할	기능
kubelet	노드에 설치된 kubelet은 마스터 노드에서 할당된 Pod을 워커 노드에서 실행하고 관리하는 주체	- Pod의 생성, 시작, 재시작, 종료 등을 담당 - 마스터 노드에게 노드 상태 및 Pod 상태를 주기적으로 보고함
kube-proxy	네트워크 프록시로, 네트워크 규칙을 설정하여 Pod 간의 통신을 지원하고 로드 밸런싱을 수행	- 서비스의 가상 IP 주소 및 포트를 실제 Pod들에 매핑 - 서비스를 통한 외부와의 통신을 관리
Container Runtime	컨테이너를 실행하고 관리하는 소프트웨어 (예시 : Docker, Containerd 등이 사용됨)	- 컨테이너 이미지를 다운로드 함 - 컨테이너의 실행, 종료, 로깅, 리소스 관리 등을 담당



Kubernetes

SECTION.02

아키텍처와 구성 요소

쿠버네티스 오브젝트란?

쿠버네티스의 오브젝트는 애플리케이션의 상태를 나타내는 엔티티(Entity)
쿠버네티스 내에서 특정 종류의 리소스를 나타냄

- 쿠버네티스의 오브젝트는 **spec(스펙, 명세)**과 **status(상태)** 등의 값을 가짐 = **desired state, current state**
- 오브젝트들은 애플리케이션을 구성하고 관리하는 데 필수적인 요소이며, 쿠버네티스 API를 통해 생성되고 관리됨

쿠버네티스의 대표적인 다섯 가지 오브젝트

Pod

Volume

Ingress

Service

Namespace

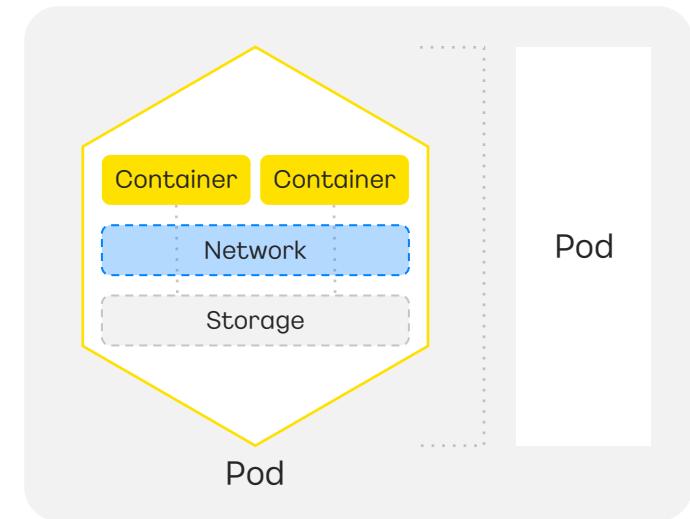
Kubernetes

아키텍처와 구성 요소

파드(Pod)란?

하나 이상의 컨테이너로 이루어진 쿠버네티스에서 생성하고 관리할 수 있는 **최소 배포 단위**

- 각 Pod는 독립적인 공간과 IP를 가짐
- 동일한 노드에서 실행되는 컨테이너 간에 네트워크 및 스토리지 공유를 가능하게 함
- 하나 이상의 컨테이너가 함께 실행되는 그룹으로 이루어짐
- 동일한 Pod 내의 컨테이너 간에는 IP주소, 포트, 파일 시스템 등이 공유됨
- 컨트롤러(deployment, replicaset, statefulset)를 이용하여 Pod의 수 관리 가능



Pod의 라이프 사이클

값	의미
Pending	- Pod을 생성 중인 상태 - 클러스터에서는 Pod를 승인했지만, 아직 실행되지 않은 상태
Running	- Pod 안 컨테이너가 모두 생성되고, 최소 하나의 컨테이너가 실행 중인 상태
Succeeded	- Pod 안 모든 컨테이너가 정상적으로 종료된 상태
Failed	- Pod 안의 컨테이너 중 정상적으로 종료되지 않은 컨테이너가 있는 상태
Unknown	- Pod의 상태를 확인할 수 없는 상태(Pod가 있는 노드와 통신 불가할 때)

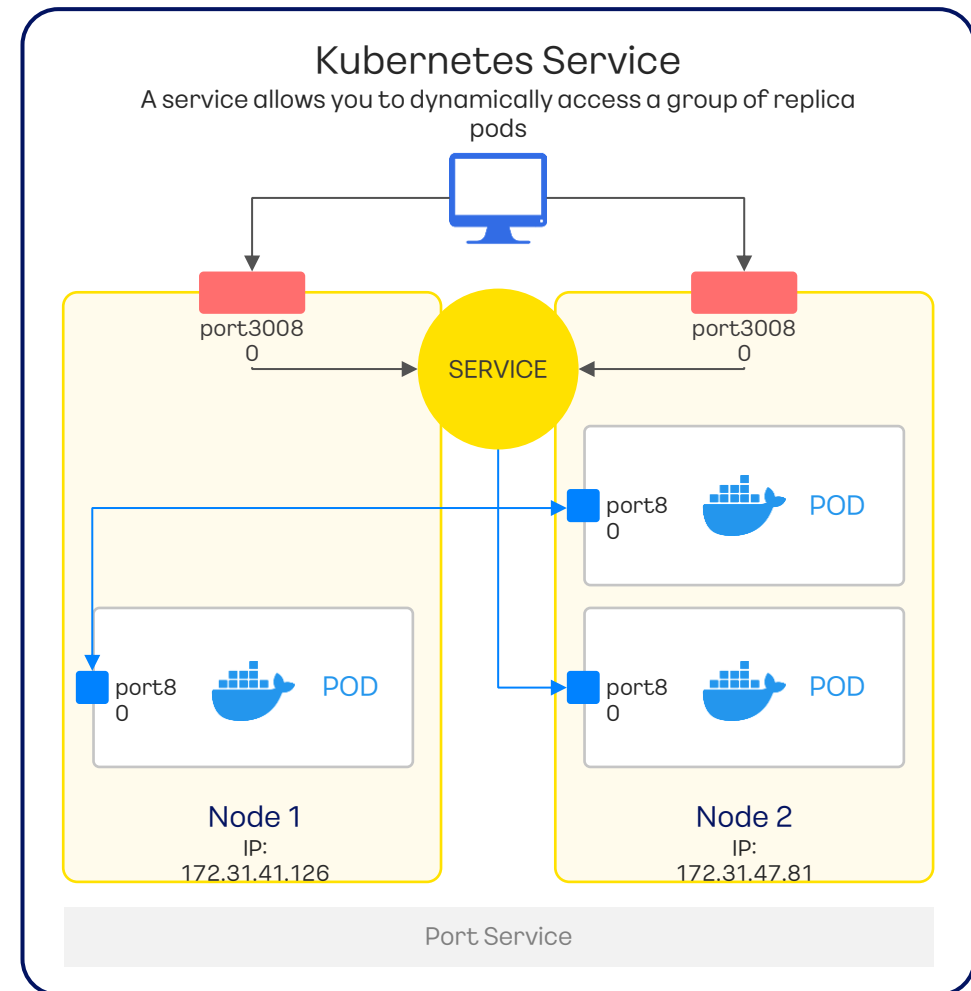
Kubernetes

아키텍처와 구성 요소

Service란?

파드들을 통해 실행되고 있는 애플리케이션을
네트워크에 노출(expose)시키는 오브젝트

- 쿠버네티스에서는 Service라는 오브젝트를 사용하여 서비스의 노출, 통신, 로드 밸런싱을 관리
- Service는 클러스터 내의 여러 Pod들에 대한 단일 진입점을 제공하고, 서비스 디스커버리와 같은 기능을 수행
- 클러스터 외부로부터 요청을 받을 수 있게 IP를 노출하는 역할을 하는 리소스



Kubernetes

SECTION.02

아키텍처와 구성 요소



Service 유형

쿠버네티스 Service는 여러 가지 유형이 있으며, 주요 유형은 다음과 같음

Cluster IP

서비스의 type을 명시적으로 지정하지 않았을 때의 기본값임.
서비스를 클러스터-내부 IP에 노출시킴(클러스터 내에서만
서비스에 도달할 수 있음)

NodePort

각 노드의 지정된 고정 포트(기본값은 30000-32767 내에서
할당)로 서비스를 노출함.
외부에서 노드의 IP 주소와 해당 포트를 통해 서비스에 접근

Load Balancer

클라우드 제공업체에서 제공하는 로드 밸런서를 사용하여
서비스를 외부에 노출

External Name

클러스터 외부에 있는 서비스에 대한 외부 DNS 주소로 매핑을 제공

Kubernetes

SECTION.02

아키텍처와 구성 요소

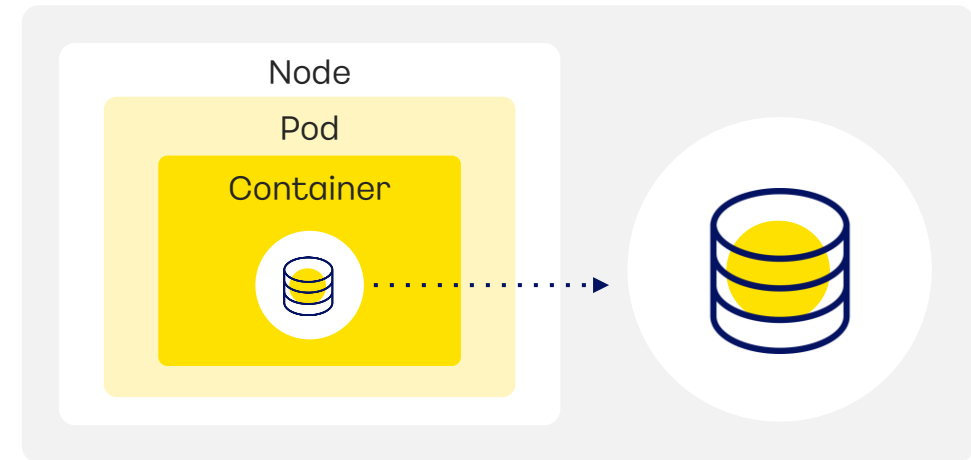
Kubernetes Volume이란?

파드의 일부분으로 정의되며 파드와 동일한 라이프사이클을 갖는 **디스크 스토리지**

- 쿠버네티스 볼륨은 파드의 구성 요소로 컨테이너와 동일하게 파드 스펙에서 정의됨
- 파드가 내 컨테이너들이 볼륨을 공유할 수 있고 **접근하려는 각 컨테이너에서 마운트하여 사용함**

주요 Volume 유형

- EmptyDir: Pod가 실행되는 동안에만 존재하는 일시적인 볼륨이며, Pod가 종료되면 데이터도 삭제
- HostPath: Pod가 실행되는 호스트의 파일 시스템에 접근할 수 있게 하는 볼륨. 파드가 삭제되어도 데이터는 유지됨.
- Persistent Volume Claim (PVC): 클러스터 내에서 지속적으로 사용 가능한 사전에 생성된 Persistent Volume(PV)를 사용
- ConfigMap 및 Secret: 설정 정보나 기밀 데이터를 Pod에 전달할 수 있게 하는 볼륨



Kubernetes

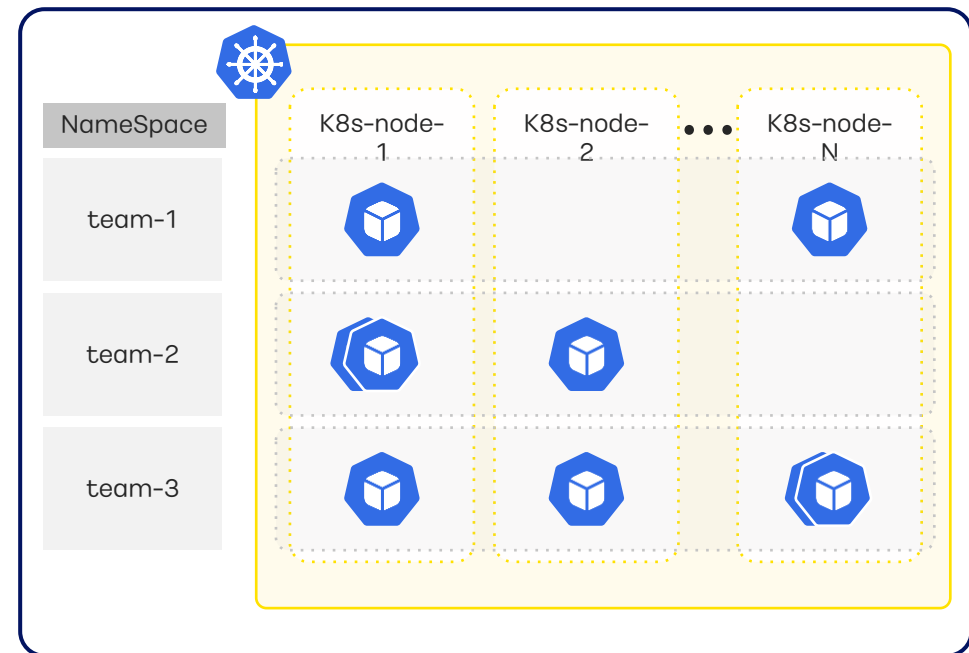
SECTION.02

아키텍처와 구성 요소

Namespace(ns)란?

쿠버네티스 클러스터 내의 **논리적인 분리 단위**

- Namespace를 지정하지 않을 시, 'default' namespace로 설정되어 있음
- 쿠버네티스에서 네임스페이스는 **클러스터 내에서 리소스를 격리**하고 다수의 팀이나 프로젝트가 동시에 작업할 수 있도록 하는 가상 클러스터를 생성하는 데 사용됨
- 네임스페이스를 사용하면 클러스터 내의 **서로 다른 그룹이나 사용자** 간에 리소스의 충돌을 방지하고, 리소스의 관리를 더 효과적으로 할 수 있음



Kubernetes

SECTION.02

아키텍처와 구성 요소

namespace의 주요 특징



리소스 격리

- 네임스페이스를 사용하면 동일한 클러스터 내에서 여러 팀이나 프로젝트 간에 리소스를 격리할 수 있음
- 한 네임스페이스에서 생성된 리소스는 다른 네임스페이스에서 생성된 리소스와 분리되어 동작함



이름 충돌 방지

- 동일한 이름을 가진 리소스라도 각 네임스페이스 내에서는 고유한 범위를 가짐
- 이는 다수의 팀이나 프로젝트가 동일한 리소스 이름을 사용할 수 있도록 함



권한 제어

- 쿠버네티스는 RBAC(Role-Based Access Control)을 제공하며, 네임스페이스를 기반으로 권한을 제어할 수 있음
- 특정 네임스페이스에 대한 권한을 팀이나 사용자에게 부여하여 해당 네임스페이스 내에서만 작업할 수 있도록 할 수 있음



환경 분리

- 각 네임스페이스는 독립된 가상 클러스터처럼 동작하므로, 환경 분리를 통해 개발, 테스트, 프로덕션 등 각 단계의 환경을 분리하여 운영할 수 있음

Kubernetes

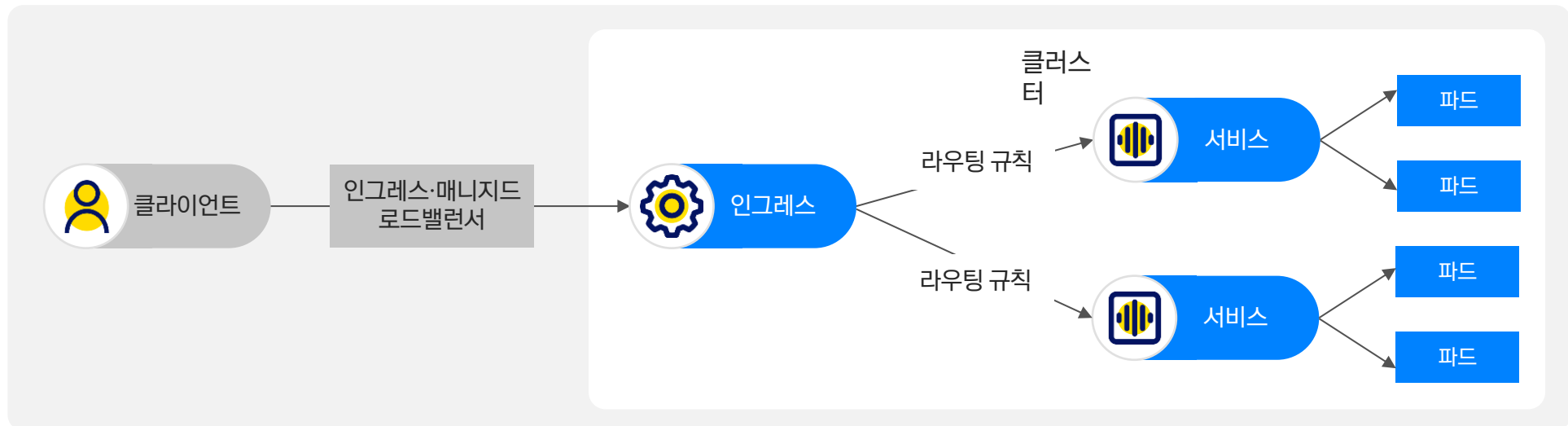
SECTION.02

아키텍처와 구성 요소

Ingress란?

클러스터에서 외부 요청을 클러스터 내의 서비스로 라우팅하는 규칙을 정의

- 클러스터 내부의 IP 주소와 포트 번호를 외부에 직접 노출하지 않고 도메인 이름을 통해서만 API 통신을 구현함으로써, 보안성을 강화할 수 있음.
- 트래픽 요청을 처리하는 규칙의 모음이며, 이를 실제로 수행하는 것은 Ingress Controller임 (e.g. nginx)
- 클러스터 외부에서 접근가능한 URL을 사용할 수 있게 하며, 트래픽을 로드밸런싱하고, SSL 인증서 처리를 하며, 도메인 기반으로 가상 호스팅을 제공하기도 함



Kubernetes

SECTION.02

아키텍처와 구성 요소

Ingress의 주요 특징

✔ 도메인 기반 라우팅

- 하나의 클러스터 IP나 외부 IP주소에 여러 서비스를 노출시켜 도메인을 기반으로 트래픽을 분배
- Ingress는 Pod에 도메인을 부여해 해당 도메인을 기반으로 트래픽을 서비스로 라우팅

✔ SSL/TLS 지원

- Ingress를 통해 SSL/TLS를 구성해 클러스터 내의 서비스에 대한 암호화된 통신 설정 가능

✔ HTTP/HTTPS 프로토콜 지원

- Ingress는 HTTP와 HTTPS 프로토콜을 지원하므로, 웹 애플리케이션의 경우 SSL 종료, 가상 호스트 기반 라우팅 등을 설정할 수 있음

✔ 다양한 Ingress 컨트롤러 지원

- 클라우드 서비스 사용 시: 로드 밸런서 서비스를 인그레스와 연동하여 사용 가능
- 클라우드 서비스 미사용 시: 인그레스 컨트롤러를 인그레스와 연동하여 사용 가능

Kubernetes

SECTION.02

아키텍처와 구성 요소

Ingress

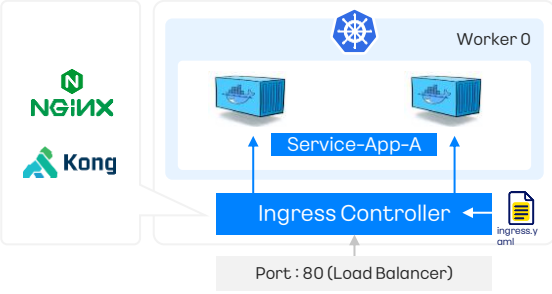


클러스터

인그레스 → 라우팅 규칙 → 서비스 → 파드

- 외부로부터 요청에 대해 TLS 관리 및 라우팅 관리를 하는 리소스
- L7 수준에서 라우팅
- SSL/TLS 통신 암호화 처리 가능
- Service와 함께 사용

Ingress Controller



Worker 0

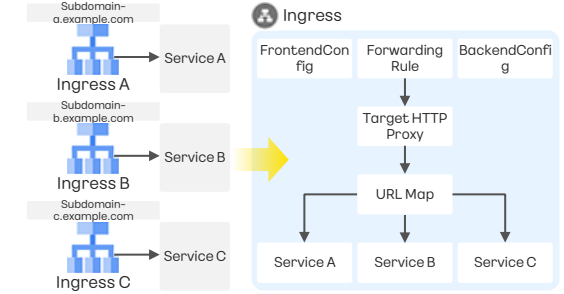
NGINX, Kong, Service-App-A, Ingress Controller

Port : 80 (Load Balancer)

- 쿠버네티스는 Ingress Controller를 제공하지 않고 Ingress API 리소스에 대한 스펙만 제공
- 해당 스펙을 처리하기 위한 Ingress Controller를 사용자가 설치해야 함
- 다양한 Ingress Controller 존재

NGINX, HAProxy, Kong, Etc.

Ingress Class



Ingress A, Ingress B, Ingress C

Service A, Service B, Service C

Ingress Class: FrontendConfig, Forwarding Rule, BackendConfig, Target HTTP Proxy, URL Map

- 하나의 클러스터에서 여러 Ingress Controller를 사용할 수 있게 만들어주는 API 리소스
- 실제 Ingress는 하나의 Ingress Class와 연결하게 됨
- 해당 Ingress는 Ingress Class가 사용하는 Ingress Controller를 통해 규칙이 정해짐

Kubernetes

SECTION.02

아키텍처와 구성 요소

레이블(label)의 용도

쿠버네티스 리소스에 레이블을 부여해 리소스를 식별하거나 그룹화 가능
쿠버네티스에서 레이블은 주로 아래 세 가지 용도에 사용 됨

버전 관리

- 애플리케이션의 버전과 같은 레이블을 지정해 버전 관리 수행
- 해당 레이블을 이용해 Pod의 버전을 지정해 롤백 및 업데이트에 사용

```
apiVersion: v1
kind: Pod
metadata:
  name: kakaoCloud-Kubernetes-Label-Exam
  labels:
    app: frontend
    version: v1.2.3
spec:
  containers:
    - name: myapp-container
      image: kakaoCloud-Kubernetes-Label-Exam:v1.2.3
```

환경구분

- 다양한 환경(개발, 스테이징, 프로덕션)에 레이블을 지정해 구분 및 관리 수행
- 다양한 환경에서 실행 중인 애플리케이션의 관리, 모니터링 단순화 가능
- 개발, 스테이징, 프로덕션과 같은 다양한 단계에서 동일한 애플리케이션 운영 및 테스트 하는 경우 유용

```
apiVersion: v1
kind: Deployment
metadata:
  name: kakaoCloud-Kubernetes-Label-Exam
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
        environment: production
    spec:
      containers:
        - name: kakaoCloud-Kubernetes-Label-Exam-container
          image: kakaoCloud-Kubernetes-Label-Exam:latest
```

서비스 식별

- 서비스와 같은 레이블을 지정해 특정 서비스를 식별 및 관리 수행
- 주로 Pod, Service, ReplicaSet, Deployment 등의 쿠버네티스 오브젝트에 부여

```
apiVersion: v1
kind: Service
metadata:
  name: kakaoCloud-Kubernetes-Label-Exam
  labels:
    app: frontend
    service: web
spec:
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```


Kubernetes

SECTION.02

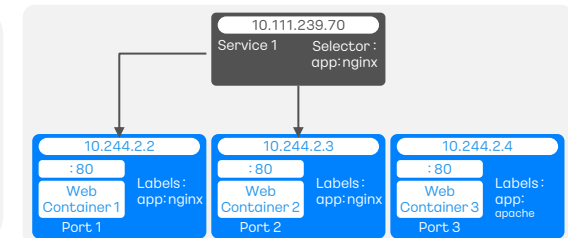
아키텍처와 구성 요소

셀렉터(Selector)

셀렉터는 레이블을 기반으로 동작하며 특정 리소스를 식별하거나 선택을 가능하게 함
쿠버네티스에서는 세 가지 유형의 셀렉터를 대표적으로 사용

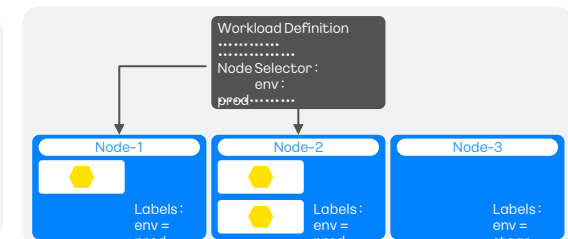
레이블 셀렉터(Label Selector)

- 리소스에 할당된 레이블을 기반으로 해당 리소스를 선택하는 방법
- 특정 애플리케이션의 Pod를 선택할 때 주로 사용



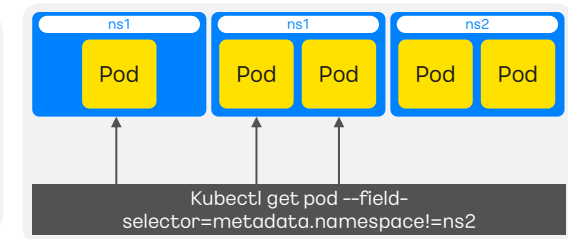
노드 셀렉터(Node Selector)

- 노드에 부여된 레이블을 기반으로 해당 노드를 선택하는 방법
- Pod가 실행 될 특정 노드를 선택할 때 사용



필드 셀렉터(Field Selector)

- 리소스의 특정 필드값을 기반으로 해당 리소스를 선택하는 방법
- Pod의 특정 엔드포인트나 Namespace 등을 선택 가능



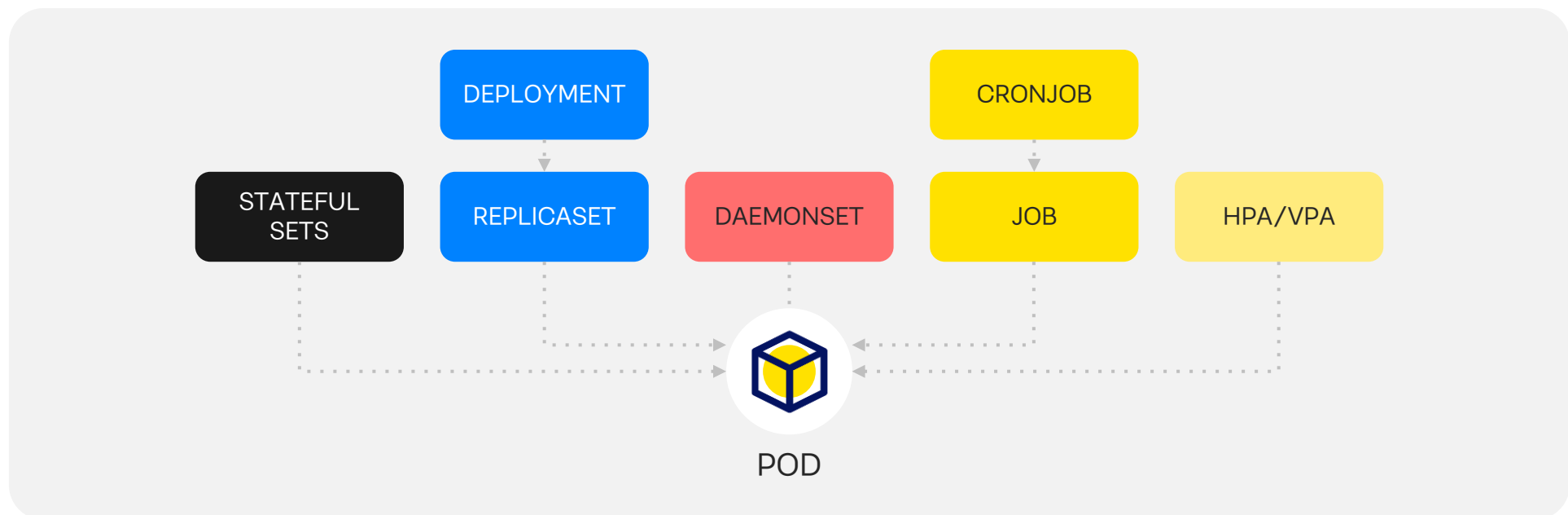
Kubernetes

아키텍처와 구성 요소

컨트롤러란?

쿠버네티스 컨트롤러는 클러스터 내의 워크로드 및 리소스를 관리하고 제어하는 역할을 하는 핵심 구성 요소

- 클러스터의 상태를 지속적으로 확인하고 원하는 상태로 유지하기 위해 작동
- 클러스터의 확장성, 가용성, 안정성 보장에 기여



Kubernetes

SECTION.02

아키텍처와 구성 요소

쿠버네티스 컨트롤러

	기능	활용 예시
ReplicaSet	- 지정된 수의 Pod 복제본이 항상 실행되고 있는지 관리 - 셀렉터를 사용하여 Pod 선택	안정적으로 지정된 수의 동일한 Pod를 유지하여 높은 가용성을 보장
Deployment	선언적으로 애플리케이션의 상태를 업데이트하고, Pod 및 ReplicaSet을 관리	애플리케이션의 새 버전을 롤아웃하거나, 롤백하거나, 확장할 때 사용
StatefulSet	순서가 지정된 배포와 스케일링, 영구적인 스토리지를 가진 Pod 관리	순서가 중요한 데이터베이스나 클러스터링된 애플리케이션 등 상태를 유지해야 하는 애플리케이션을 배포할 때 사용
DaemonSet	클러스터의 모든 노드(또는 일부 노드)에 Pod의 복사본을 실행	클러스터의 각 노드에 로깅, 모니터링 등의 에이전트를 배포하고자 할 때 사용
Job 및 CronJob	일회성 또는 주기적인 작업을 실행하도록 관리	배치 처리, 데이터 처리, 주기적인 작업 등을 위해 사용

카카오클라우드

Container Pack

- Kubernetes Engine

SECTION.03

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

카카오클라우드 Kubernetes Engine이란?

Kubernetes Engine 서비스

- 카카오클라우드에서 제공하는 **관리형 Kubernetes 서비스**
- 번거로운 수작업 없이 복잡한 설정이나 관리 절차 없이 Kubernetes 클러스터를 운영할 수 있도록 지원
- **VPC 기반**으로 Kubernetes 클러스터를 손쉽게 생성 및 관리
- Kubernetes 오픈소스 **표준을 준수**
- 신규 버전 출시 후 안정화 및 테스트를 거쳐 **클러스터 업데이트를 신속하게 제공**
- **Kubernetes RBAC와 IAM을 통합 사용**하여 클러스터 접근을 안전하게 관리할 수 있음.



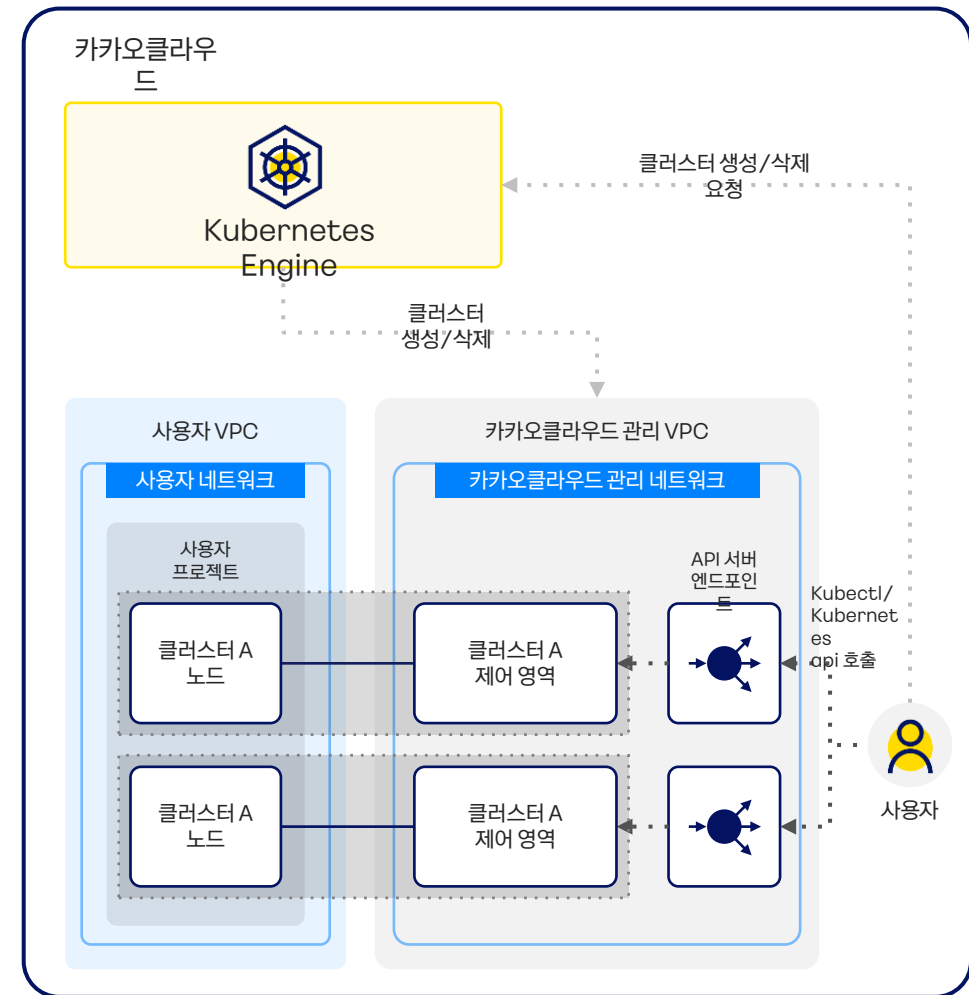
Kubernetes Engine

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

Kubernetes Engine 아키텍처

- Kubernetes Engine의 클러스터를 위한 네트워크는 **카카오클라우드 관리 VPC**와 **사용자 VPC 영역**으로 구분됨
- 쿠버네티스의 마스터 노드는 카카오클라우드가 관리하는 VPC 내에 있으며, 카카오 엔터프라이즈에서 전적으로 관리함.
- 사용자가 마스터 노드의 관리와 유지보수에 신경 쓸 필요가 없다는 것을 의미함.
- 사용자는 자신의 VPC 내에 배포된 워커 노드와 그 위에서 실행되는 애플리케이션에 집중할 수 있음.
- 사용자는 Kubernetes의 API 서버 인터페이스를 통해 클러스터를 관리할 수 있고, 필요한 모든 작업을 Kubectl이나 Kubernetes API를 사용하여 실행할 수 있음.
- 이러한 구조는 **사용자에게 클러스터 관리의 복잡성을 줄이고**, 인프라에 대한 걱정 없이 **애플리케이션 개발에 더 집중할 수 있는 환경**을 제공함.



카카오클라우드 Container Pack - Kubernetes Engine

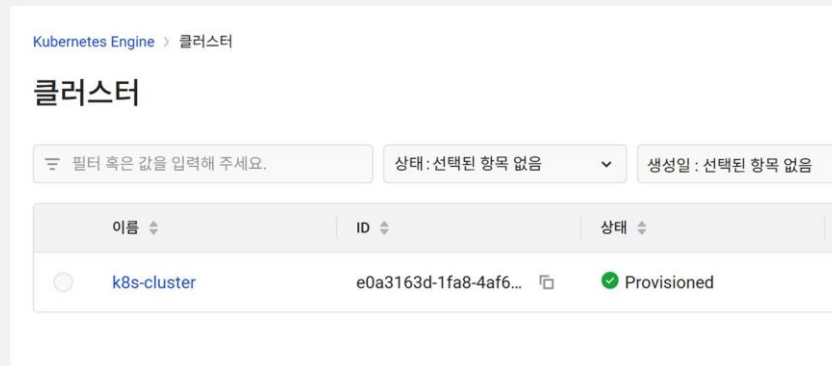
SECTION.03

Kubernetes Cluster 사용자 접근 방식

제어 영역에 접근하는 두 가지 방식

- 사용자는 카카오클라우드에서 제공하는 Kubernetes Engine을 관리하기 위해 아래 두 가지 방식 중 선택할 수 있음

카카오클라우드 콘솔 UI 접근



Kubectl 및 HTTP 요청을 통한 접근

```
munseongha@munseonghau-MacBookPro ~ % kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
host-172-30-0-17    Ready    <none>    22h   v1.26.6
host-172-30-32-246  Ready    <none>    22h   v1.26.6
```

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

클러스터 생성 및 관리

카카오클라우드 Kubernetes Engine를 통해 클러스터를 간편하게 생성 및 관리가능

카카오클라우드 콘솔의 쿠버네티스 관리 기능

클러스터 목록 보기

노드 풀 생성/삭제 및 정보 확인

클러스터 설정 보기

노드 정보 확인

클러스터 검색

클러스터 자동 축소 설정

클러스터 상세보기

클러스터 삭제

클러스터 세부 정보 확인

클러스터 업데이트

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

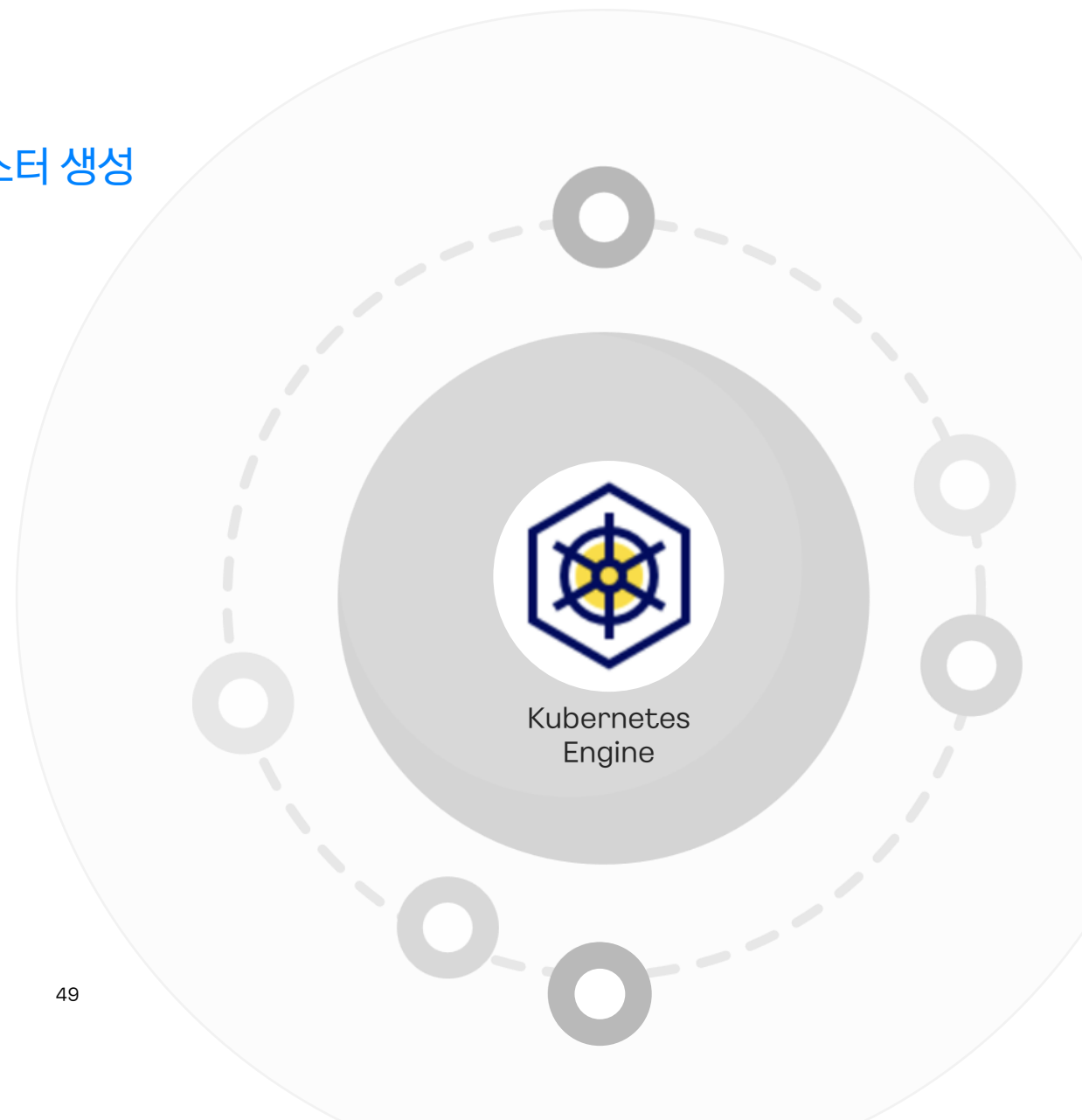
클러스터 생성 및 관리

카카오클라우드 Kubernetes Engine 클러스터 생성

- Kubernetes Engine의 Console UI를 통해 클러스터 생성

클러스터 생성 시 정의해야 할 정보

- 클러스터 기본 설정
- 클러스터 네트워크 영역
- 클러스터 CNI



카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

클러스터 생성 및 관리

클러스터 생성

카카오클라우드 콘솔에서 클러스터 생성 가능

클러스터 생성 시 입력할 정보

The screenshot shows the 'Kubernetes Engine' console with the '클러스터 생성' (Create Cluster) form. The form is divided into several sections: '기본 정보' (Basic Info), '클러스터 네트워크 설정' (Cluster Network Settings), and 'CNI'. The '기본 정보' section includes '클러스터 이름' (Cluster Name), 'Kubernetes 버전' (Kubernetes Version), and '클러스터 네트워크 설정' (Cluster Network Settings). The '클러스터 네트워크 설정' section includes 'VPC' and 'Subnet'. The 'CNI' section includes 'CNI' and '서비스 IP CIDR 블록' (Service IP CIDR Block). A detailed note about CNI configuration is visible at the bottom of the form.

구분	설명	
기본 설정	클러스터 이름	- 영어 소문자, 숫자, 하이픈(-)을 사용해 4~20자 이내로 작성 - 시작 문자는 영어 소문자만 가능하며 하이픈(-)으로 끝낼 수 없음
	Kubernetes 버전	- 클러스터에서 사용할 Kubernetes 버전 선택 - Kubernetes Engine에서 제공하는 지원 버전에 대한 자세한 설명은 지원 정보 참고
클러스터 네트워크 설정	VPC	프로젝트에 공유된 VPC 중 클러스터를 배포할 VPC 선택
	Subnet	- 프로젝트에 공유된 서브넷 중 클러스터를 배포할 서브넷 선택 - 노드 풀 생성 시, 클러스터 Network에서 설정한 서브넷 중 노드가 실행될 서브넷 선택 - Multi-AZ를 지원하는 kr-central-2 리전에서는 노드 풀의 네트워크를 <u>서로 다른 AZ의 Subnet으로 다중 선택하여 가용성을 높일 수 있음</u> (kr-central-2 리전에서 클러스터의 Subnet은 각 AZ 별로 최소 1개 이상의 Subnet이 설정되어야 함)
	클러스터 엔드포인트 액세스 선택	- 퍼블릭 또는 프라이빗 엔드포인트 액세스 선택 - 퍼블릭 : 외부 인터넷에서 연결이 가능한 클러스터 엔드포인트 - 프라이빗 : VPC 내부 네트워크에 연결이 제한된 클러스터 엔드포인트
CNI	CNI	클러스터에서 사용할 CNI 선택 (Calico, Cilium CNI 플러그인 지원)
	서비스 IP CIDR 블록	- 클러스터의 서비스 객체가 수신할 IP를 지정 - Default: 네트워크 설정에서 지정한 서브넷의 CIDR 범위를 따름
	파드 IP CIDR 블록	- 파드가 수신할 IP를 지정 - Default: 네트워크 설정에서 지정한 서브넷의 CIDR 범위를 따름

카카오클라우드 Container Pack - Kubernetes Engine

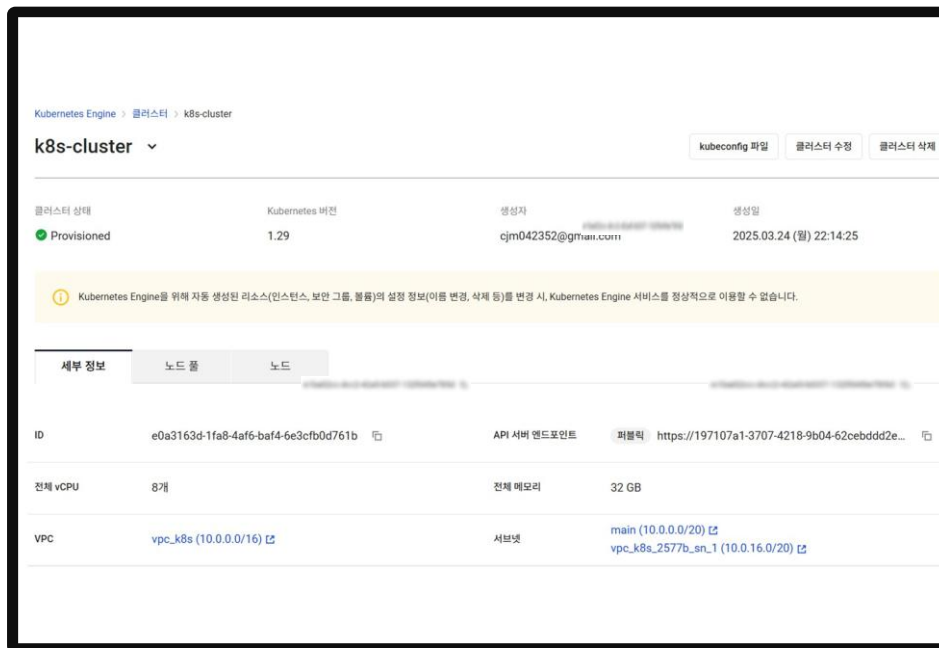
SECTION.03

클러스터 생성 및 관리

클러스터 세부 정보 조회

Kubernetes Engine 서비스를 통해 클러스터 설정 및 조회 가능

클러스터 세부정보 화면



구분	설명
ID	클러스터의 고유 ID 값
API 서버 엔드포인트	클러스터 API를 호출할 엔드포인트
전체 vCPU	클러스터 내 워커 노드들의 vCPU 총합
전체 메모리	클러스터 내 워커 노드들의 메모리 총합
VPC	클러스터가 배포된 VPC
서브넷	클러스터가 배포된 서브넷 - kr-central-2 리전의 클러스터는 가용성을 높이기 위해 여러 가용 영역의 서브넷이 다중 설정된 경우 해당하는 모든 서브넷이 표시

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

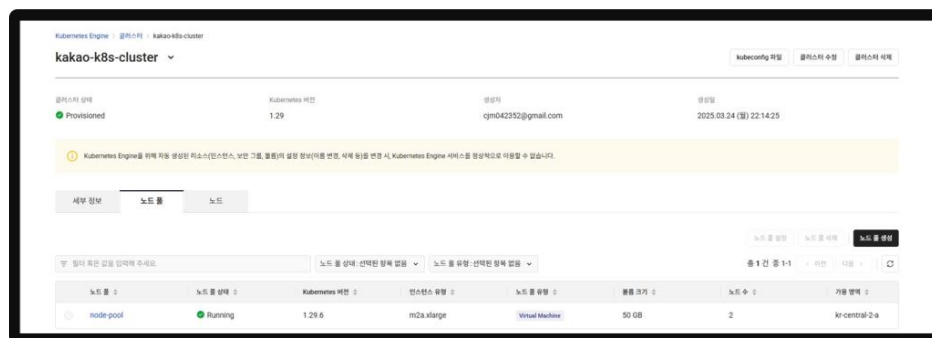
노드 풀 정보 확인

노드 풀이란?

- 노드 풀은 동일한 인스턴스 유형을 가지는 노드 그룹이며, Kubernetes Engine에서는 노드를 노드 풀 단위로 관리함
- 클러스터 생성 후 별도로 노드 풀을 생성해야 함
- 서로 다른 인스턴스 유형을 가지는 다수의 노드 풀을 클러스터에 추가할 수 있음 (최대 5개)
- 노드 풀의 각 노드에는 노드 풀의 이름을 값으로 가지는 kakaoi.io/kke-nodpool이라는 Kubernetes 노드 레이블이 설정되어 있음

클러스터 내 노드 풀과 노드 풀에 속한 노드의 정보를 확인할 수 있음

노드 풀 정보 확인



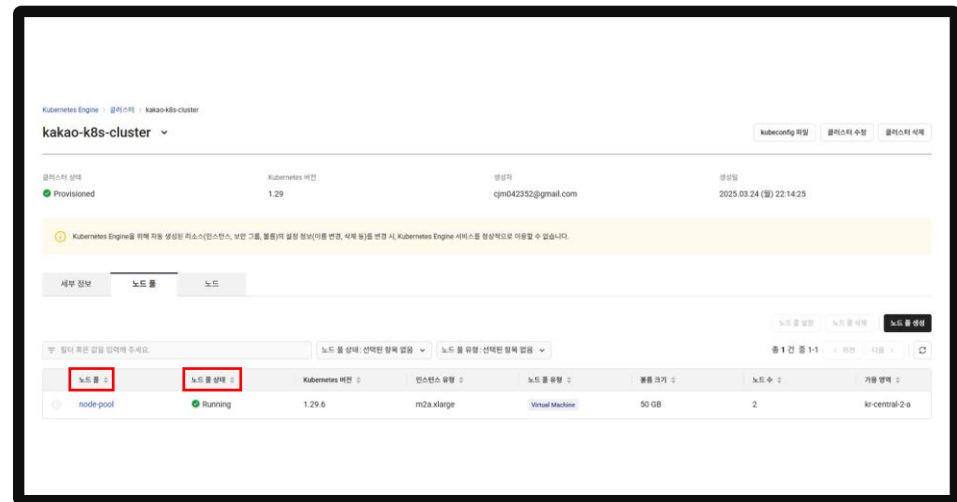
카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

노드 풀 정보 확인

노드 풀 정보 확인에 나타나는 정보들

구분	설명
노드 풀 필터	필터를 통해 노드 풀 검색
[노드 풀 만들기] 버튼	클릭 시 신규 노드 풀 생성
노드 풀	노드 풀의 이름 및 설명 - 노드 풀 이름 클릭 시, 노드 풀 상세 페이지로 이동
노드 풀 상태	노드 풀의 상태 - Running : 노드 풀 정상 실행 중(노드 상태와는 무관) - Running (Scheduling Disable) : 노드 풀의 모든 노드가 스케줄링 차단된 상태 - ScalingUp : 노드 개수 확장 중 - ScalingDown : 노드 개수 축소 중 - Deleting : 노드 풀 삭제 중 - Failed : 사용자의 개입이 필요한 실패 상태



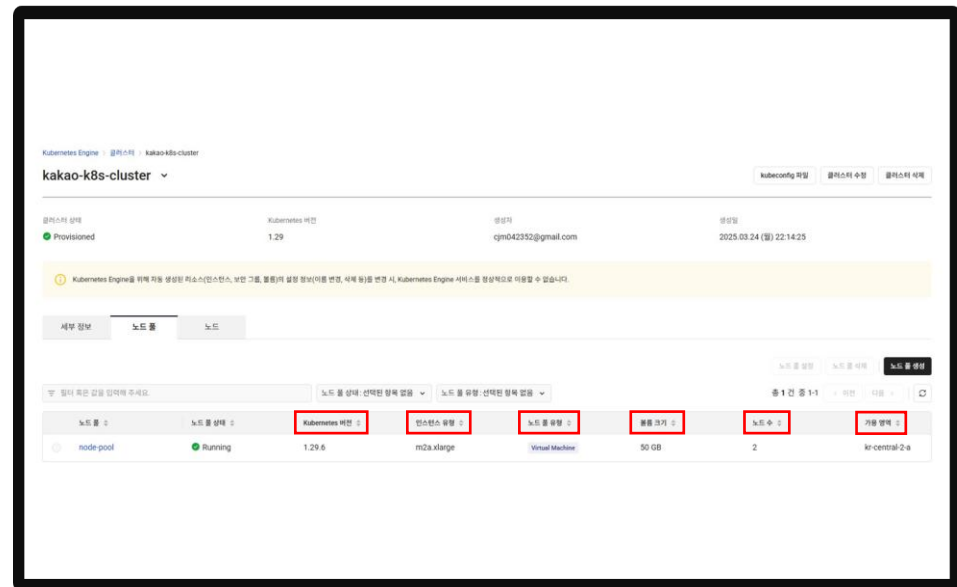
카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

노드 풀 정보 확인

노드 풀 정보 확인에 나타나는 정보들

구분	설명
Kubernetes 버전	노드 풀의 노드가 현재 사용 중인 Kubernetes 버전
인스턴스 유형	노드 풀의 노드에서 사용하는 인스턴스 유형
노드 풀 유형	노드가 동작하는 인스턴스 종류
볼륨 크기	노드 풀의 노드에서 사용하는 인스턴스의 볼륨 크기
노드 수	현재 노드 풀에 속한 노드 수
가용 영역	노드 풀에서 사용하는 서브넷의 가용 영역 정보



카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

노드 풀 생성 및 관리

✓ 노드 풀 생성

- 노드 풀은 **동일한 인스턴스 유형을 가지는 노드 그룹**이며, Kubernetes Engine에서는 노드를 **노드 풀 단위로 관리**
- 노드 풀에서 특정 노드만 삭제하는 것은 불가능하며 노드 스케줄링 차단은 가능함
- 기본적으로 클러스터당 노드풀 5개까지 생성 가능

기본 정보 항목

항목	설명
노드 풀 유형	생성할 노드 풀 유형을 선택 - Bare Metal Server 유형은 kr-central-2 리전만 지원
기본 설정	노드 풀의 기본 정보 설정 - 노드 풀 이름 : 영어 소문자, 숫자, 하이픈(-)을 사용해 4 ~ 20자 이내로 작성, 시작 문자는 영어 소문자만 가능하며 하이픈(-)으로 끝날 수 없음 - 노드 풀 설명(선택) : 60자 이내로 노드 풀 설명 작성
이미지	노드에 사용할 이미지를 1개 선택 - 노드 풀 타입에 따라 선택 가능한 이미지가 다름

노드 풀 유형

Virtual Machine

워커 노드를 Virtual Machine 인스턴스에서 실행합니다.

GPU

워커 노드를 GPU 인스턴스에서 실행합니다.

Bare Metal Server

워커 노드를 Bare Metal 단독 서버에 실행합니다.

기본 설정

노드 풀 이름

영어로 시작하며, 영문 소문자, 숫자, 하이픈(-)만 입력 (4~20자)

설명

60자 이내로 작성

이미지

필터 혹은 값을 입력해 주세요.

총 1 건 중 1-1

< 이전

다음 >

이름	아키텍처
☒ Ubuntu 22.04 (VM)	x86_64

① 노드 인스턴스에 사용할 이미지를 선택합니다.

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

노드 풀 생성 및 관리

✓ 노드 풀 생성 설정

- 노드 풀의 기본 정보(Instance 타입, Volume, 노드 수) 설정 가능

기본 정보 항목

항목	설명
인스턴스 유형	노드 풀의 <u>인스턴스 유형</u> 선택
Volume	인스턴스의 볼륨 타입과 크기를 설정, 현재 볼륨 타입은 SSD 타입으로 고정되어 있고, 볼륨 크기는 30 ~ 5,120GB 내에서 지정 가능
노드 수	노드 풀의 노드 개수 설정 (기본 노드 수는 0, 노드 풀 당 최대 노드 수는 50개까지 가능)

인스턴스 유형

> 조건 검색

인스턴스 유형을 선택해 주세요.

볼륨

SSD 50 GB

노드 수

0 - +

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

노드 풀 생성 및 관리

✓ 노드 풀 생성 설정

- 노드가 실행되는 VPC, 서브넷, 키 페어 선택

네트워크 설정 항목

항목	설명
VPC	클러스터의 VPC와 동일하며, 별도 설정 불가
서브넷	클러스터 생성 시 선택한 서브넷 중, 해당 노드 풀의 노드가 실행될 서브넷을 선택 - Multi-AZ를 지원하는 kr-central-2 리전의 경우, 노드 풀의 network를 서로 다른 AZ의 Subnet으로 다중 선택하여 가용성을 높일 수 있음

노드 풀 네트워크 설정 ⓘ

VPC
vpc_k8s (10.0.0.0/16) ↕

서브넷
인스턴스 유형을 선택해 주세요. ↕

보안 그룹 (선택) ⓘ
선택된 항목 (0) ↕

선택한 보안 그룹의 상세 정책을 아래에서 확인할 수 있습니다.

[인바운드/아웃바운드 규칙](#) ▾

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

노드 풀 생성 및 관리

리소스 기반 오토 스케일링 설정

노드 풀의 리소스 사용에 따라 노드의 수를 자동으로 확장 또는 축소하는 기능

노드 가용 리소스가 부족할 시 자동으로 노드를 추가하고, 노드 리소스 사용률이 일정 수준 이하라면 노드를 줄임

리소스 기반 오토 스케일링 설정

리소스 기반 오토 스케일링 설정 정보

구분	항목	설명
리소스 기반 오토 스케일링	원하는 노드 수	현재 노드 풀의 노드 수로, 변경 가능함
	최소 노드 수	자동으로 노드 수를 축소할 때 가능한 최소 노드 수
	최대 노드 수	자동으로 노드 수를 확장할 때 가능한 최대 노드 수
자동 축소 규칙	자동 축소 임계치 조건	노드 수를 자동으로 축소하는 기준인 노드 리소스(CPU/Memory) 사용량의 임계치 - 입력 가능 범위 : 1 ~ 100 - 기본값: 50%
	임계치 지속 시간	노드의 리소스 사용량이 자동 축소 임계치 조건 이하로 유지되는 일정 시간 기준 - 입력 가능 범위: 1 ~ 86400 (초), 1 ~ 1440 (분) - 기본값: 10분
	자동 축소 모니터링 제외 시간	노드 자동 증설 후, 자동 축소 노드 모니터링 시 일정 시간 동안 포함하지 않는 시간 - 입력 가능 범위: 1 ~ 86400 (초), 1 ~ 1440 (분) - 기본값: 10분

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

노드 풀 생성 및 관리

✓ 예약 기반 오토 스케일링 설정

노드 풀의 노드 개수를 **예약 시점에 조정**하는 기능
트래픽 증가 또는 감소 시간을 예측하여 노드 풀의 노드 개수 조정 자동화 가능

예약 기반 오토 스케일링 설정

예약 기반 오토 스케일링 설정 정보

구분	설명
이름	예약 기반 오토스케일의 규칙 이름
규칙	원하는 노드 수 - 설정된 시간에 원하는 노드 수를 설정 반복 설정 - 규칙이 주기에 따라 반복 동작하기 위한 설정 [한 번], [매일], [매주], [매월]
시작 일시	규칙이 동작하는 시작 일시 - 시작 일시에 따라 반복 시점이 결정됩니다.
실행 예정일	시작 일시에 따른 가장 가까운 실행 예정일을 표시합니다.

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

노드 상세 정보

노드(Node)

- 노드풀은 비슷한 구성을 공유하는 노드들의 집합이며, 각 노드는 특정 노드풀의 일부임
- 노드들은 사용자가 설정한 VPC의 서브넷에 위치함

노드 정보 확인

The screenshot shows the 'kakao-k8s-cluster' page in the Kakao Cloud console. It displays the cluster's status as 'Provisioned' and provides details about the Kubernetes version (1.29), the user (rosco0611@naver.com), and the creation time (2025.03.25 (Thu) 20:13:44). Below this, there's a warning message about the Kubernetes Engine. The '노드' (Nodes) tab is selected, showing a table of nodes. The table has columns for Node ID, Node Name, Node Pool, IP Address, Instance Type, and Creation Time. Two nodes are listed: 'host-10-0-64-15' and 'host-10-0-0-123', both in 'Running' state, belonging to the 'node-pool', with IP addresses '10.0.64.15' and '10.0.0.123' respectively, using 'kr-central-2-b' instance type, and created 35 minutes ago.

노드 ID	노드 이름	노드 풀	호스트아이피 IP	가용 영역	파드 스케줄링	가용 시간
host-10-0-64-15	Running	node-pool	10.0.64.15	kr-central-2-b	허용	35분
host-10-0-0-123	Running	node-pool	10.0.0.123	kr-central-2-a	허용	35분

구분	설명
노드	노드 이름 및 설명 - 노드 이름 클릭 시 노드 상세 페이지로 이동
노드 상태	노드 상태 정보 - Running : 노드가 준비되어 실행 중 - Running (Scheduling Disable) : 해당 노드로 신규 스케줄링이 차단된 상태(이미 할당되어 실행 중인 파드와는 무관) - Provisioned : 노드 프로비저닝 완료 - Deleted : 노드 삭제 완료 - Pending : 노드 프로비저닝 준비 중 - Provisioning : 노드 프로비저닝 중 - Deleting : 노드 삭제 중 - Failed : 사용자의 개입이 필요한 실패 상태
노드 풀	노드가 속한 노드 풀의 정보 - 노드 풀 클릭 시, 노드 풀 상세 페이지로 이동
사설 IP	노드의 사설 IP 정보
AZ	노드가 실행되는 Subnet의 AZ 정보
생성 요청 시간	노드의 생성이 요청된 후 경과된 시간으로, 노드의 생성일을 의미하지 않음

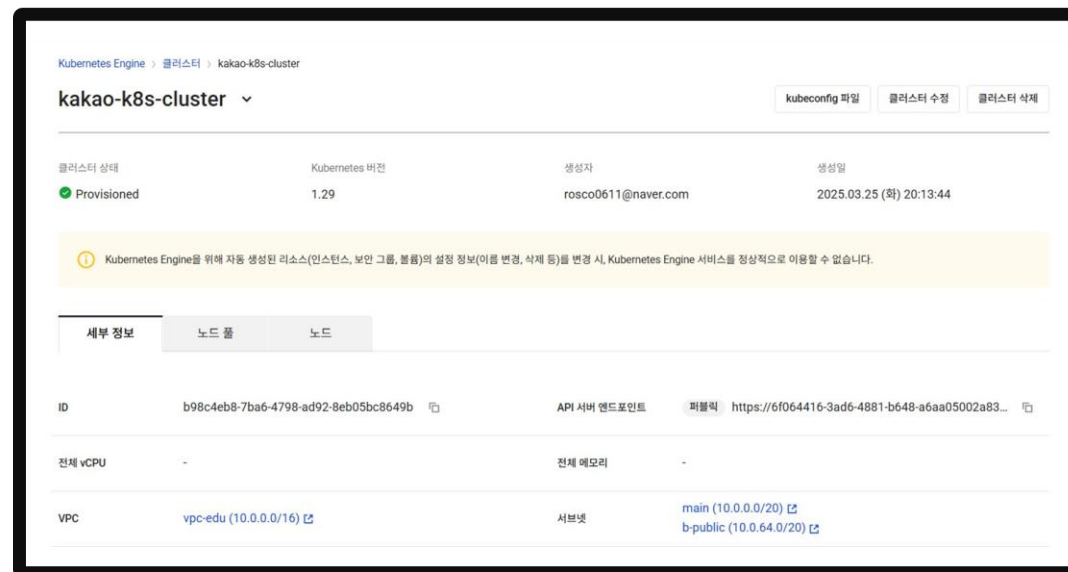
카카오클라우드 Container Pack – Kubernetes Engine

SECTION.03

KakaoCloud의 Kubernetes Engine 특징

간편한 클러스터 관리

- Kubernetes 클러스터를 효율적으로 관리하기 위한 **최적화된 메뉴 구성 및 기능**과 **노드 풀을 이용한 노드 그룹 관리 기능**을 제공
- VM 서비스와의 긴밀한 연계로 **다양한 VM 플레이버를 지원**하고 손쉬운 인스턴스 관리가 가능



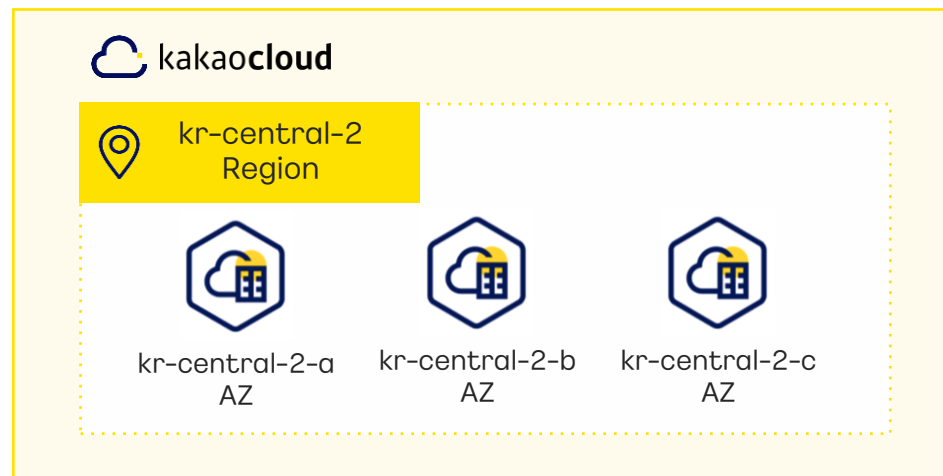
카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

KakaoCloud의 Kubernetes Engine 특징

Multi-AZ를 통한 고가용성 지원

- Kubernetes Engine의 클러스터와 노드를 **Multi-AZ**에서 실행하여 **높은 수준의 가용성을 제공**
- 클러스터와 노드가 여러 AZ(가용 영역)에서 실행되며, 특정 AZ에 문제가 발생해도 다른 AZ에서 정상적으로 실행되어 **서비스 중단 없는 고가용성을 제공**



클러스터 만들기

기본 설정

클러스터 이름
kakao-k8s-cluster

클러스터 설명 (선택)
60자 이내로 작성

Kubernetes 버전
1.26

클러스터 Network 설정 ⓘ
[관리 페이지](#)

VPC
vpc-1 (172.30.0.0/16)

Subnet
선택한 항목 (3)
kr-central-2-a (2), kr-central-2-b (1)

Q. |검색어를 입력해 주세요|

☒ 전체 선택	
☒ main (172.30.0.0/20)	kr-central-2-a
☒ main-b (172.30.32.0/20)	kr-central-2-b
☒ private (172.30.16.0/20)	kr-central-2-a

클러스터 Network 설정 ⓘ
[관리 페이지](#)

VPC
vpc-1 (172.30.0.0/16)

Subnet
선택한 항목 (1)
kr-central-2-a (1)

클러스터의 Subnet은 각 AZ별 1개 이상 설정되어야 합니다.

main ✕ 전체 선택

* 클러스터 생성 시 Subnet은 각 AZ별 1개 이상 설정되어야 함

카카오클라우드 Container Pack - Kubernetes Engine

SECTION.03

KakaoCloud의 Kubernetes Engine 특징

다양한 서비스팩 연동

- 카카오클라우드의 다른 서비스들(IAM, Load Balancer, VPC, Container Registry)과 함께 연동하여 사용
- Container Registry와의 연동을 통해 Helm Chart와 컨테이너 이미지를 간단하게 배포



IAM

보안 주체 및 액세스 사용
- 카카오클라우드의 IAM

- RBAC(Role-Based Access Control)
기반 권한 관리
- 권한이 있는 사용자들에게만 시스템 접근을
허용하는 방식



Load Balancer

카카오클라우드의
Load Balancer를 통한
부하 분산



VPC

카카오클라우드의 VPC를 활용한
노드 격리를 통해 안전한 서비스
환경 제공



Container Registry

클라우드 환경에서 컨테이너
이미지를 클라우드에 안전하게
보관하고 관리할 수 있는 이미지
저장소

카카오클라우드 Container Pack - Kubernetes Engine

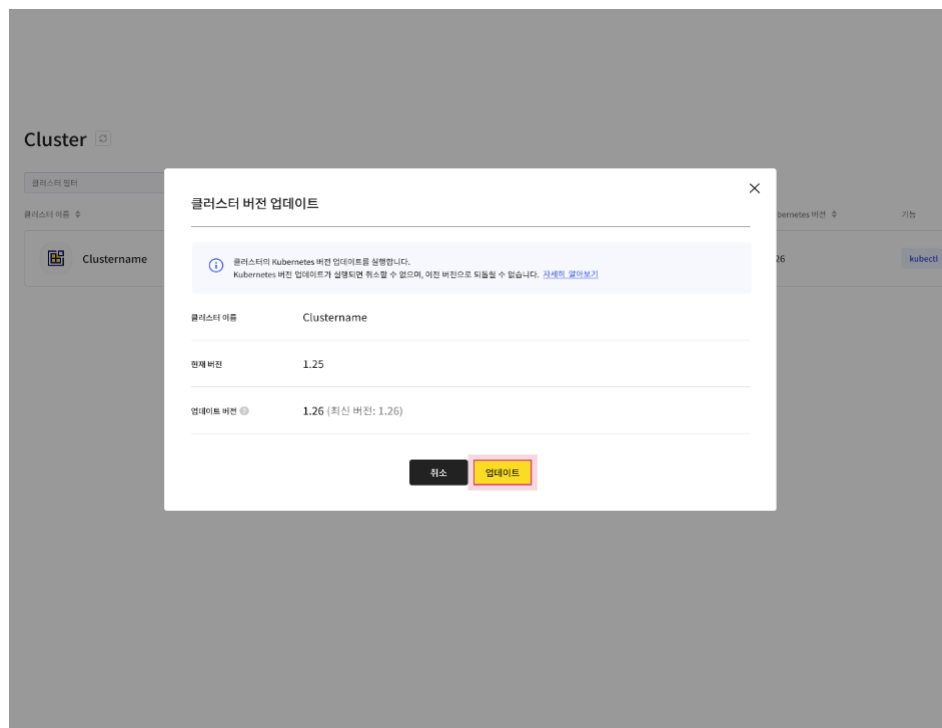
SECTION.03

KakaoCloud의 Kubernetes Engine 특징

Kubernetes 최신 버전을 사용하여 손쉬운 마이그레이션 가능

카카오클라우드 콘솔 UI를 통한 클러스터 업데이트 지원

지원 Kubernetes 버전(2025.4 기준)



Kubernetes 버전	Kubernetes Engine 지원 시작일	클러스터 신규 생성	생성된 클러스터 사용
1.29	2024년 12월 20일	가능	가능
1.28	2024년 5월 28일	가능	가능
1.27	2024년 3월 18일	불가능	가능
1.26	2023년 8월 16일	불가능	가능
1.25	2023년 8월 16일	불가능	가능
1.24	2023년 2월 1일	불가능	가능
1.23	2023년 7월 7일	불가능	가능
1.22	2023년 7월 7일	불가능	가능

Lab2 : Kubernetes Engine Cluster 생성

SECTION.02



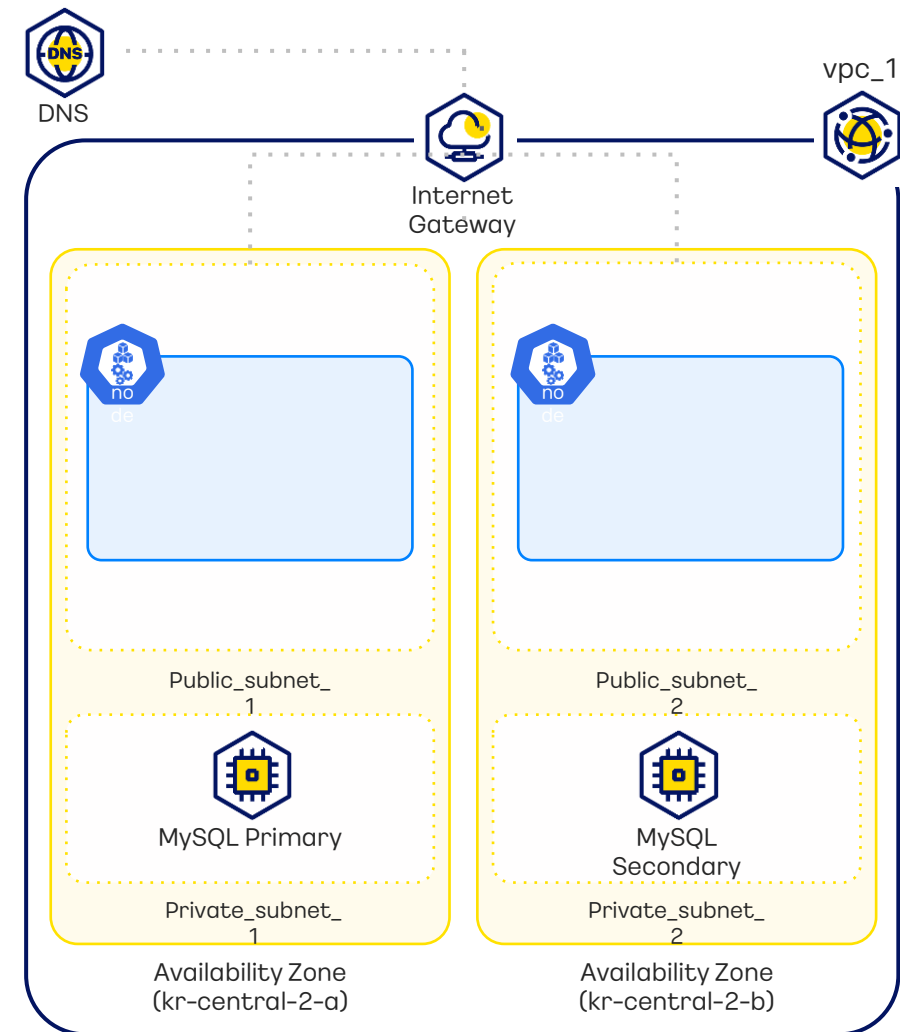
(실습 교재 13 page, [Github Link](#))

Lab

KakaoCloud Kubernetes Engine Cluster 생성에 대한 실습입니다.

목차

- 1 클러스터 생성 (Demo로 진행)
- 2 노드 풀 생성



Lab3 : Kubernetes Engine Cluster 관리 VM 생성 실습

SECTION.03



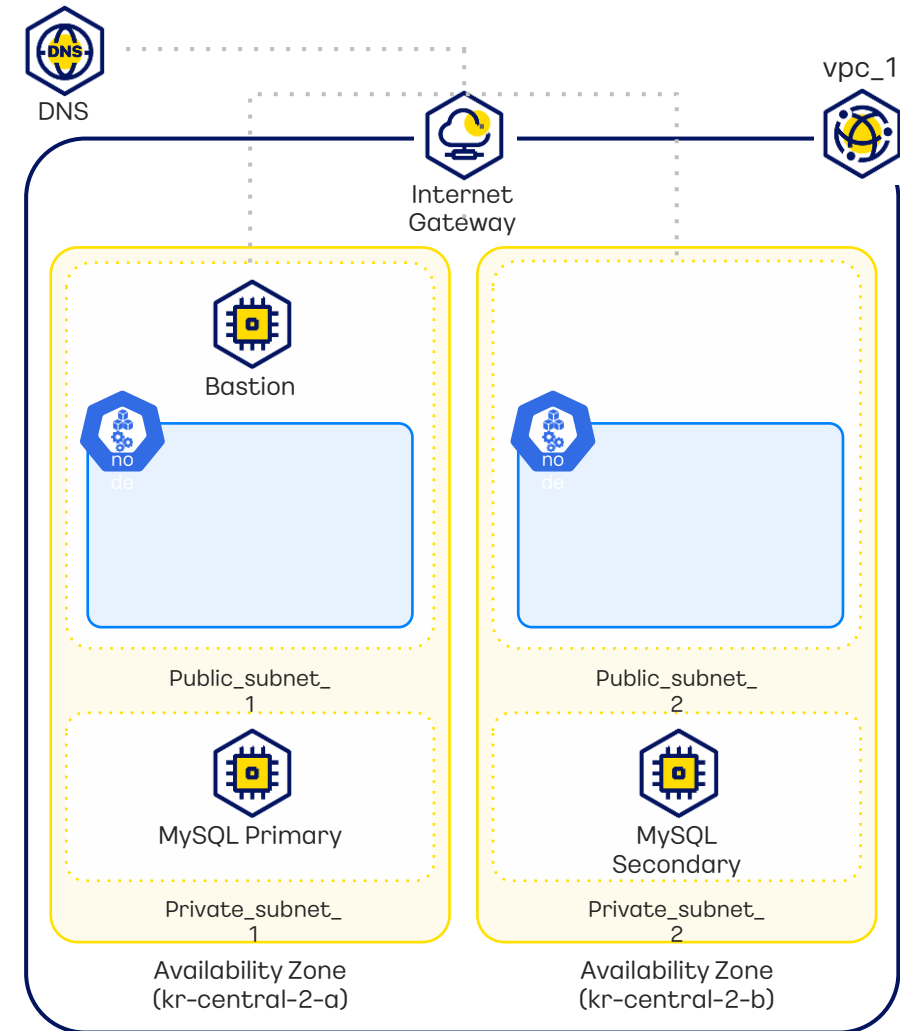
(실습 교재 21 page, [Github Link](#))

Lab

Kubernetes Engine Cluster를 관리하기 위한 Bastion VM 인스턴스를 생성합니다. VM 내부에 클러스터를 사용하기 위한 패키지들을 설치하고, 실습 환경을 구성합니다.

목차

- 1 Bastion VM 인스턴스 생성
- 2 Bastion VM 인스턴스를 통해 클러스터 확인



카카오클라우드

Container Pack

– Container Registry

SECTION.04

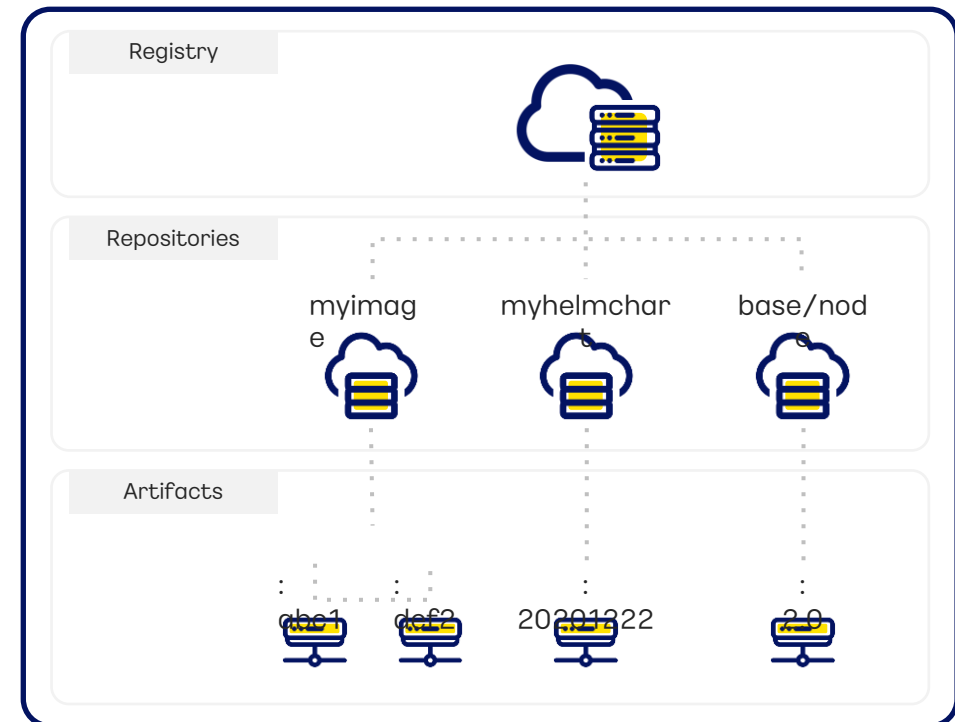
카카오클라우드 Container Pack - Container Registry

SECTION.04

Container Registry 란?

카카오클라우드의 Container Registry 서비스는 클라우드에서 도커 컨테이너 이미지를 안전하게 저장하고 관리하는 이미지 저장소

- 카카오클라우드의 **Kubernetes Engine**과 연동되어 컨테이너 기반 애플리케이션과 서비스 컨테이너 이미지를 효율적으로 배포할 수 있도록 지원
- **IAM 기반으로 사용자 권한을 관리**하므로 안정성과 보안성을 유지하면서 개발 및 운영 프로세스를 최적화
- **표준 Docker 이미지를 지원**하여 호환성과 이식성이 우수하며, 이미지 버전 관리와 업데이트가 쉽게 이루어짐



카카오클라우드 Container Pack - Container Registry

SECTION.04

Container Registry의 특징

✓ 기본적인 컨테이너 이미지 관리

- 리포지토리별 이미지 배포 및 저장
- 검색 기능을 통한 이미지 간편 조회 및 공유
- 컨테이너 이미지를 개발, 테스트, 스테이징, 프로덕션 등 여러 환경으로 배포 용이
- 다양한 환경 내 일관된 이미지 사용

✓ 이미지 이력 관리

- 이미지 버전을 나타내는 태그를 사용하여 이미지 관리
- 이미지의 변경 이력과 업데이트 추적 및 기록
- 필요 시 변경 전 이미지 버전으로 복구 가능
- Git과 유사한 브랜치 및 플로우 관리 제공

✓ 이미지 취약점 스캔

- 이미지 업로드 또는 업데이트 시 취약점 자동 스캔
- 패키지, 종속성, 런타임 환경 검사
- 취약점 정보와 보고서 형태로 스캔 결과 제공
- 취약점 발견 시 경고 및 알림 전송으로 신속한 조치 가능

✓ IAM 접근 제어

- IAM 역할이 프로젝트 관리자 권한인 경우, 리포지토리 권한을 설정하여 다른 사용자에게 리포지토리의 권한을 부여할 수 있음.
- 리포지토리 멤버는 이미지 Push, Pull 가능
- 리포지토리 뷰어는 이미지 Pull만 가능

카카오클라우드 Container Pack - Container Registry

SECTION.04

Container Registry 리포지토리 생성

카카오클라우드 콘솔을 통해 리포지토리를 생성 가능

리포지토리 만들기

공개 여부

☒ 비공개 IAM 또는 리포지토리 권한을 가진 사용자만 접근할 수 있습니다.
 ☐ 공개 URI를 아는 사용자는 누구나 이미지를 Pull 할 수 있습니다.

리포지토리 이름

영어로 시작, 소문자(a-z), 숫자(0-9), '-'만 입력 (4~20자)

리포지토리를 생성한 후에는 해당 리포지토리 이름을 수정할 수 없습니다.

리포지토리 설명 (선택)

100자 이내로 작성

태그 덮어쓰기

☒ 가능 동일한 태그를 사용하는 이미지를 Push 할 때 이미지 태그가 덮어쓰기 됩니다.
 ☐ 불가 동일한 이미지 태그를 사용해서 이미지를 Push 할 수 없습니다.

이미지 스캔

☒ 자동 리포지토리에 이미지가 Push 될 때 자동으로 이미지 스캔을 실행합니다.
 ☐ 수동 리포지토리에 저장된 이미지를 수동으로 스캔해야 합니다.

구분	설명
공개여부	- 리포지토리 공개 여부 설정 - 비공개 : IAM과 리포지토리 권한에 따라 제한된 사용자만 접근 가능 - 공개 : URI를 아는 사용자 누구나 이미지를 PULL 가능
리포지토리 이름	- 리포지토리 이름 입력(추후 수정 불가) - 영어 소문자/숫자/- (하이픈) 사용 가능 - 영어 소문자로 시작하며, 4~20자 이내 - -(하이픈)으로 끝낼 수 없으며, 연속적인 --(하이픈)사용 불가능
리포지토리 설명	- 리포지토리에 대한 설명 100자 이내로 제한
태그 덮어쓰기	- 리포지토리의 태그와 중복된 태그 이름을 사용하는 이미지를 PUSH 할 때, 덮어쓰기 기능 유무를 설정 - 가능, 불가능 타입 설정
이미지 스캔	- 리포지토리에 이미지가 Push 될 때 스캔 자동/수동 여부 설정 - 자동, 수동 타입 설정

카카오클라우드 Container Pack - Container Registry

SECTION.04

Container Registry 이미지 URI 구조 및 리소스 쿼터

이미지 URI 구조

- `https://{프로젝트 고유 ID}.{리전}.kcr.dev/{리포지토리 이름}/{이미지 이름}:{태그 이름}`

리소스 쿼터

- 리포지토리는 프로젝트 당 최대 1,000개
- 이미지는 리포지토리 당 최대 10,000개
- 태그는 이미지 당 1,000개

카카오클라우드 Container Pack - Container Registry

SECTION.04

Container Registry 이미지 관리

카카오클라우드에서 제공하는 커맨드를 통한 이미지 관리 예시

- 카카오클라우드에서 제공하는 커맨드 예시를 통해 로컬에 있는 이미지를 태깅 후 Push하거나 리포지토리에 있는 이미지를 Pull 가능
- 단, 해당 기능은 도커의 설치와 API 키 발급이 선행되어야 함

```
munseongha@munseonghau1-MacBookPro ~ % docker push kakaocloud.kr-central-2.kcr.dev/kakao-cloud/kakao_image_exam:1.0
The push refers to repository [kakaocloud.kr-central-2.kcr.dev/kakao-cloud/kakao_image_exam]
484a9239a160: Pushed
60881a8cc619: Pushed
855ace31b698: Pushed
5b5efa882b3f: Pushed
dd8e71bdf24c: Pushed
98eecd8e6843: Pushed
57131b473ab1: Pushed
ab43e2b85120: Pushed
3dd67e0995d4: Pushed
03486b619b8a: Pushed
1.0: digest: sha256:a426f450342b186f5cb797c4c2294e25be36a18e9167f76a349fc3923a2528ce size: 2411
```

이미지 1

사용 이력

이미지 필터

🔍

이미지 이름

📏

kakao_image_exam

커맨드 보기

자세한 내용은 [사용자 가이드](#)를 참고하세요

- 1. 도커 로그인**
레지스트리를 사용하기 위해서는 도커를 설치하고 API 키를 발급받아야 합니다. 자세한 내용은 [사용자 액세스 키](#) 가이드 문서를 참고해 주세요.
사용자 액세스 키를 생성해서 발급 받은 액세스 키 ID와 보안 키를 아래 명령어에 사용해서 로그인을 할 수 있습니다.

`docker login kakaocloud.kr-central-2.kcr.dev --username {사용자 액세스 키 ID} --password {사용자 액세스 보안 키}`

복사
- 2. 이미지 Push를 위한 태깅**
리포지토리에 로컬 이미지를 Push하기 위해서는 아래 샘플 커맨드를 이용해 이미지를 태깅합니다.

`docker tag {소스 이미지} kakaocloud.kr-central-2.kcr.dev/kakao-cloud/{이미지 이름}:{태그 이름}`

복사
- 3. 이미지 Push**
위에서 태깅한 이미지를 아래 샘플 커맨드를 이용해 리포지토리에 Push 합니다.

`docker push kakaocloud.kr-central-2.kcr.dev/kakao-cloud/{이미지 이름}:{태그 이름}`

복사
- 4. 이미지 Pull**
아래 샘플 커맨드를 이용해 리포지토리에 있는 특정 이미지를 Pull 할 수 있습니다.

`docker pull kakaocloud.kr-central-2.kcr.dev/kakao-cloud/{이미지 이름}:{태그 이름}`

복사

카카오클라우드 Container Pack - Container Registry

SECTION.04

Container Registry 이미지 관리

카카오클라우드 콘솔 UI를 통한 이미지 관리 기능

- 리포지토리에 있는 이미지의 세부 정보, 이미지 태그, 태그 히스토리, 사용이력 등을 조회 가능
- 리포지토리에 있는 이미지의 만료기한 변경 가능

The image displays four screenshots of the Kakao Cloud Container Registry console interface, illustrating various image management capabilities.

Top Left Screenshot: Shows the 'kakao-image-exam' repository details. The '세부 정보' (Detailed Information) tab is selected, displaying the image URI, status (Active), and a table of push events. The table includes columns for '세부 정보', '태그 1', '태그 히스토리', and '사용이력'.

Top Right Screenshot: Shows the 'kakao-image-exam' repository details. The '태그 히스토리' (Tag History) tab is selected, displaying a table of tag history with columns for '날짜' (Date), '태그 히스토리', and '복구' (Restore).

Bottom Left Screenshot: Shows the 'kakao-image-exam' repository details. The '사용이력' (Usage History) tab is selected, displaying a table of usage history with columns for '날짜' (Date), '사용이력', and '사용자' (User).

Bottom Right Screenshot: Shows the 'kakao-image-exam' repository details. The '태그 히스토리' (Tag History) tab is selected, displaying a table of tag history with columns for '날짜' (Date), '태그 히스토리', and '복구' (Restore).

Lab4 : Container Image 만들기

(실습 교재 43 page, [Github Link](#))

SECTION.04

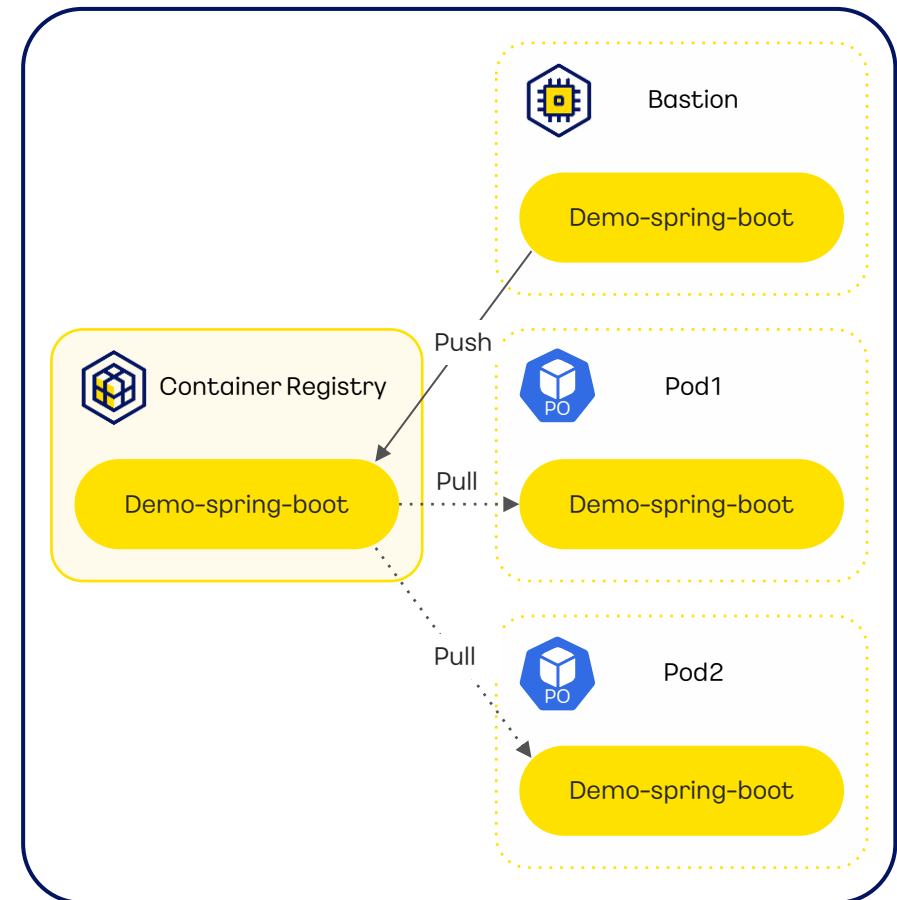


Lab

Spring Boot 프로젝트를 생성해 간단한 웹 페이지를 생성합니다.
생성한 프로젝트를 Docker Image 파일로 만들어 KakaoCloud
Container Registry에 업로드하는 실습을 진행합니다.

목차

- 1 Container Registry 생성
- 2 Spring 애플리케이션을 Container 이미지로 만들기
- 3 Container Registry에 이미지 업로드



Kubernetes Engine의 클러스터 관리 방법

SECTION.05

Kubernetes Engine의 클러스터 관리 방법

SECTION.05

Console UI를 통한 관리

콘솔을 통한 Kubernetes 클러스터 관리

카카오클라우드 콘솔의 쿠버네티스 관리 기능

클러스터 목록 보기

노드 풀 정보 확인

클러스터 설정 보기

노드 정보 확인

클러스터 검색

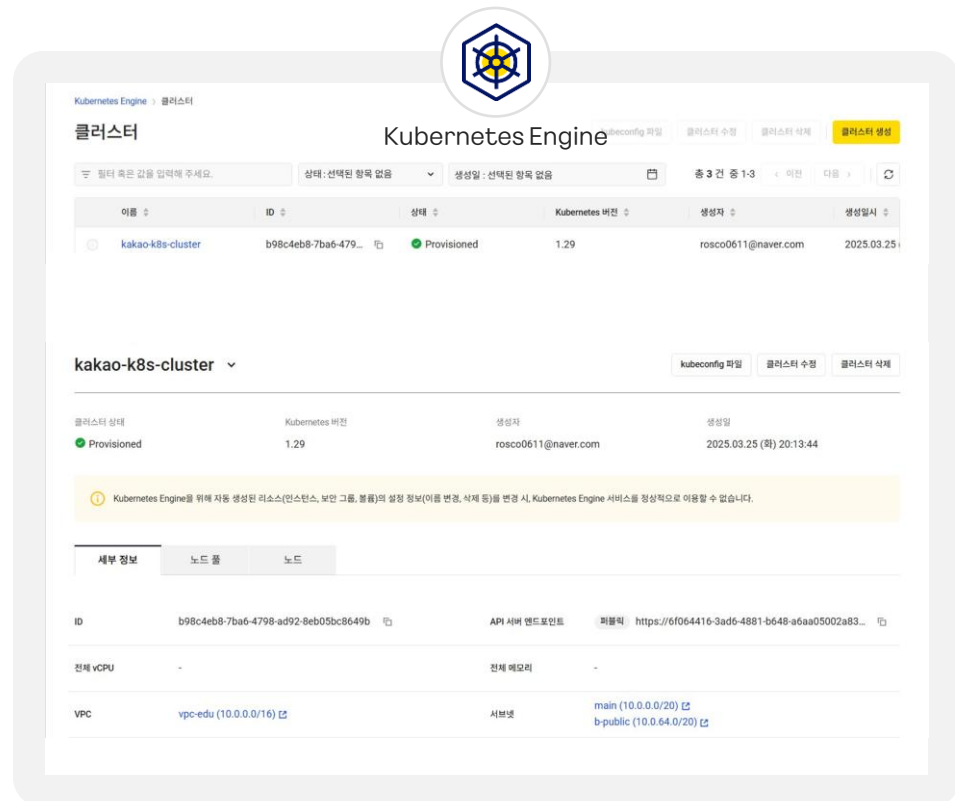
클러스터 자동 축소 설정

클러스터 상세보기

클러스터 삭제

클러스터 세부 정보 확인

클러스터 업데이트



Kubernetes Engine의 클러스터 관리 방법

SECTION.05

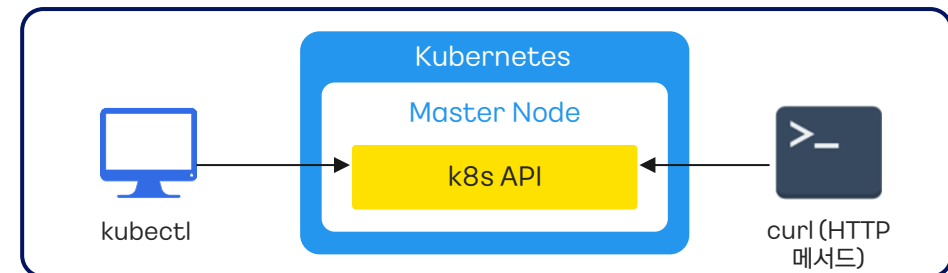
HTTP와 gRPC의 사용

API를 통한 Kubernetes 클러스터 관리

Kubernetes의 API 통신 시 사용되는 프로토콜

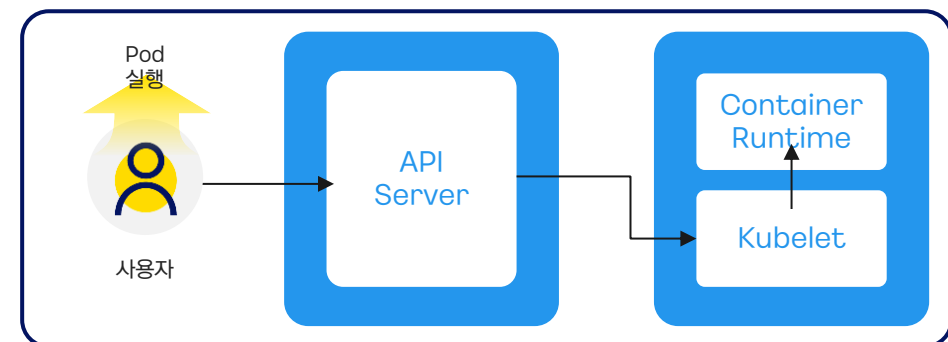
✓ HTTP

- Kubernetes는 HTTP를 통해 API 서버(REST API)와 상호작용하여 클러스터를 관리
- 1. Kubernetes에서 공식적으로 지원하는 kubectl CLI
- 2. curl을 사용한 직접적인 HTTP 요청



✓ gRPC 프로토콜

- API Server <-> Kubelet
사용자로부터 온 Pod 생성 및 관리 요청을 Kubelet에게 전달
- Kubelet<->Container Runtime
Kubelet은 API Server로부터 온 Pod 실행 및 관리 요청을 컨테이너 런타임(Docker 등)에게 보내 컨테이너를 실행 및 관리



Kubernetes Engine의 클러스터 관리 방법

SECTION.05

Kubectl이란?

Kubectl이란?

Kubernetes 클러스터와 상호 작용하기 위한 커맨드 라인 도구

다양한 kubectl 명령어 기능

POD 관리

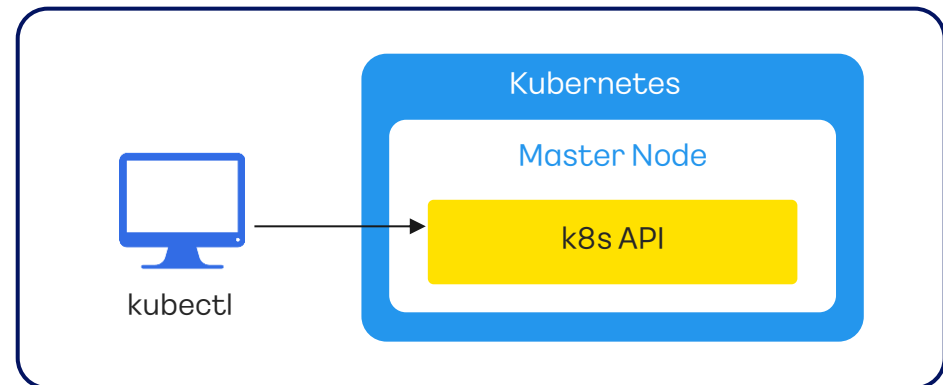
인그레스 관리

리소스 관리

서비스 관리

네임스페이스 관리

컨트롤러 관리 등



```
munseongha@munseonghau-MacBookPro ~ % kubectl get nodes
NAME                 STATUS    ROLES    AGE   VERSION
host-172-30-0-17     Ready    <none>   40m   v1.26.6
host-172-30-32-246   Ready    <none>   40m   v1.26.6
```

Kubernetes Engine의 클러스터 관리 방법

SECTION.05

IaC를 통한 관리

IaC(Infrastructure as a Code)란?

코드를 통해 인프라 구성 요소를 자동화하여 관리하고 프로비저닝함

IaC를 지원하는 대표적인 도구

✓ Terraform

- Terraform은 하시코프(HashiCorp)에서 오픈소스로 제공하는 클라우드 Infrastructure 자동화 도구
- **다양한 인프라 지원**: 다양한 클라우드 및 온프레미스 인프라를 지원하여 다양한 환경에서 리소스 관리 가능
- **선언적 코드**: HCL(Hashicorp Configuration Language)로 정의된 코드를 테라폼이 자동으로 구현
- **상태 관리**: 인프라의 현재 상태를 추적하고 관리하여 변경 사항을 안전하고 예측 가능하게 배포



✓ Ansible

- Ansible은 패키지 설치, 설정 파일 수정 등의 환경 구성을 위한 자동화 도구
- **에이전트리스(Agentless)**: 타겟 노드에 별도의 에이전트 설치 없이 SSH 또는 WinRM을 통해 작업을 수행
- **선언적 언어**: YAML 문법을 사용하여 인프라와 애플리케이션의 원하는 상태를 정의
- **플레이북(Playbooks)**: 플레이북을 통해 구성 관리, 애플리케이션 배포 등의 자동화 작업을 정의



Kubernetes Engine의 클러스터 관리 방법

SECTION.05

Helm Chart

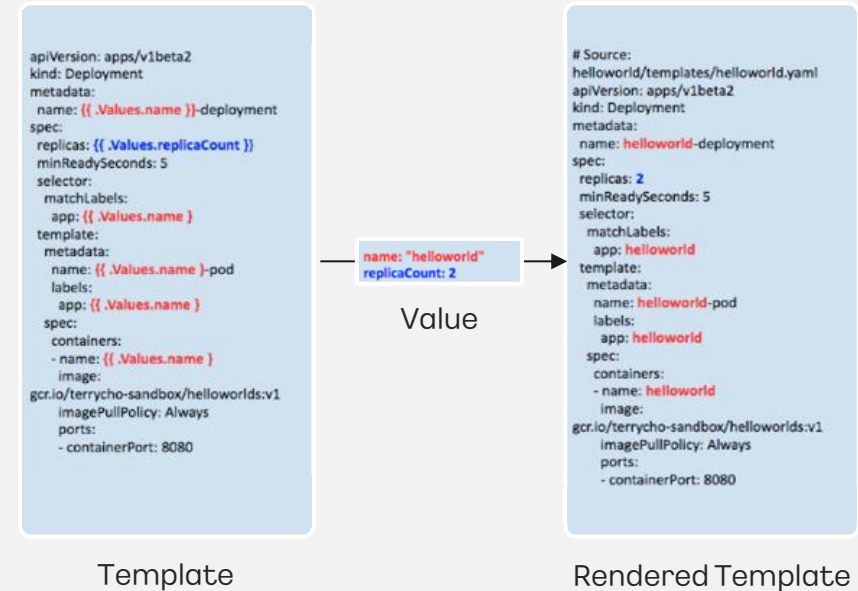
Helm Chart 란?

Kubernetes 리소스를 하나로 묶어 여러 개의 YAML 파일을 손쉽게 관리 가능한 패키지 관리 도구



✓ Helm Chart의 특징

- 애플리케이션의 모든 Kubernetes 리소스를 **하나의 패키지로 관리**
- 템플릿 엔진을 사용해 **YAML 파일을 동적으로 생성**할 수 있는 기능이 포함
- 애플리케이션의 **버전을 쉽게 관리하고 롤백**할 수 있는 기능 **제공**
- 환경 설정에 대한 값을 **변수화(Value)**시켜 각각의 환경에 대해 다른 구성을 사용 가능하게 함
- Helm Hub와 같은 공유 리포지토리를 통해 커뮤니티에 공유 가능



Kubernetes Engine의 클러스터 관리 방법

SECTION.05

Kubernetes Engine의 IAM 역할 관리

사용자 역할 별 클러스터/노드 권한

권한	프로젝트 관리자 (Admin)	프로젝트 멤버 (Member)	프로젝트 리더 (Reader)
프로젝트 멤버 관리	✓		
클러스터 생성	✓	✓	
클러스터 삭제	✓	✓	
클러스터 설정	✓	✓	
클러스터 조회	✓	✓	✓
노드 풀 추가	✓	✓	
노드 풀 삭제	✓	✓	
노드 풀 설정	✓	✓	
노드 풀 조회	✓	✓	✓
노드 삭제	✓	✓	
노드 조회	✓	✓	✓

Kubernetes Engine의 클러스터 관리 방법

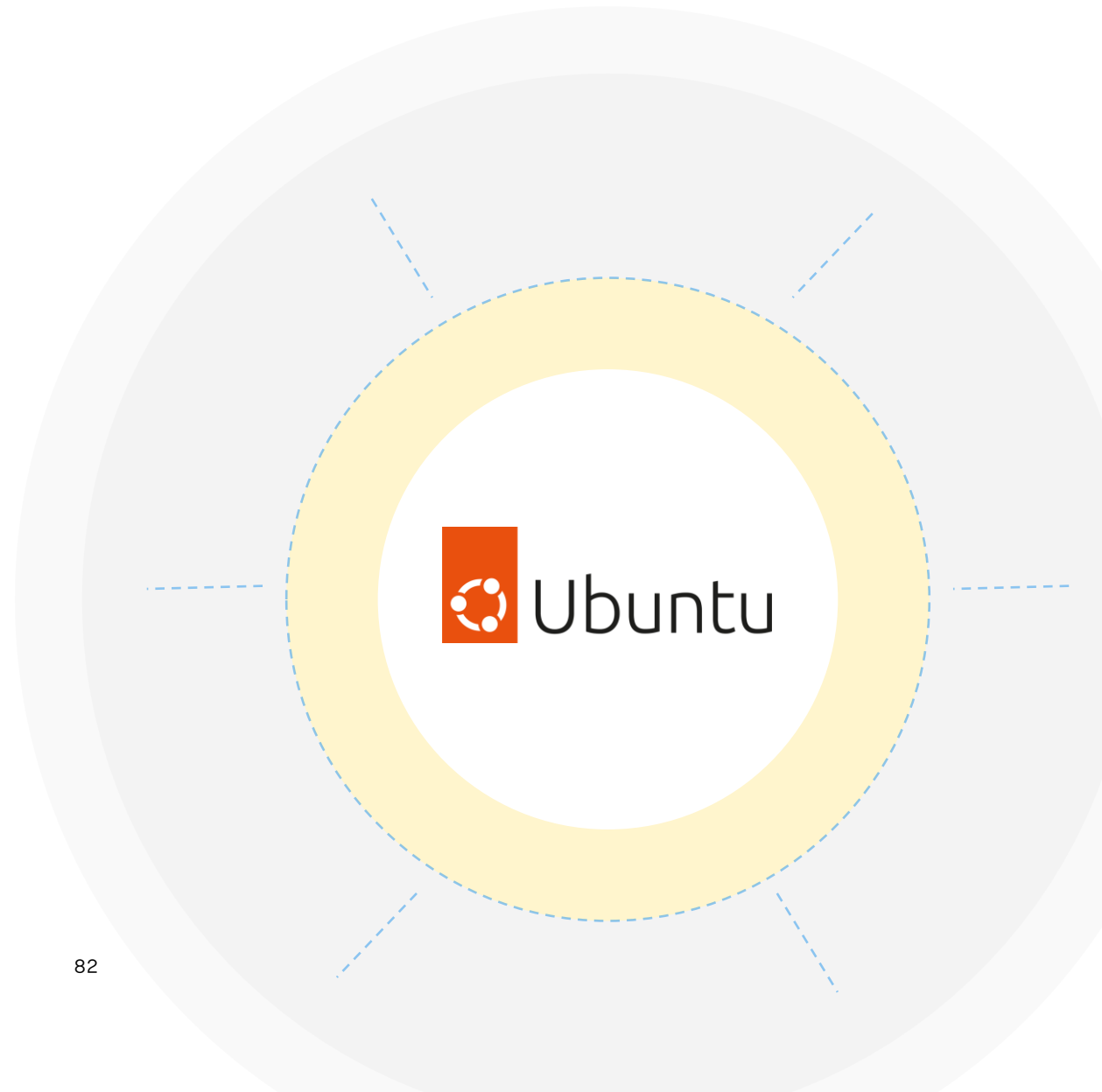
SECTION.05

Kubernetes Engine의 지원 노드 이미지

K8s Engine 지원 노드 이미지

Kubernetes Engine에서 제공하는 워커 노드 이미지는 **Ubuntu** 기반이며, Kubernetes 버전에 최적화된 OS 이미지를 제공

- Container Runtime(e.g. Docker)과의 호환
- 노드 구성에 더 적합한 배포판의 업데이트 및 지원
- 커뮤니티 및 최신 버전의 지원
- 최신 노드 이미지 출시 시 노드 업데이트를 참고하여 노드 이미지 업데이트 가능



Kubernetes Engine의 사용자 정의 설정 및 웹서버 배포

SECTION.06

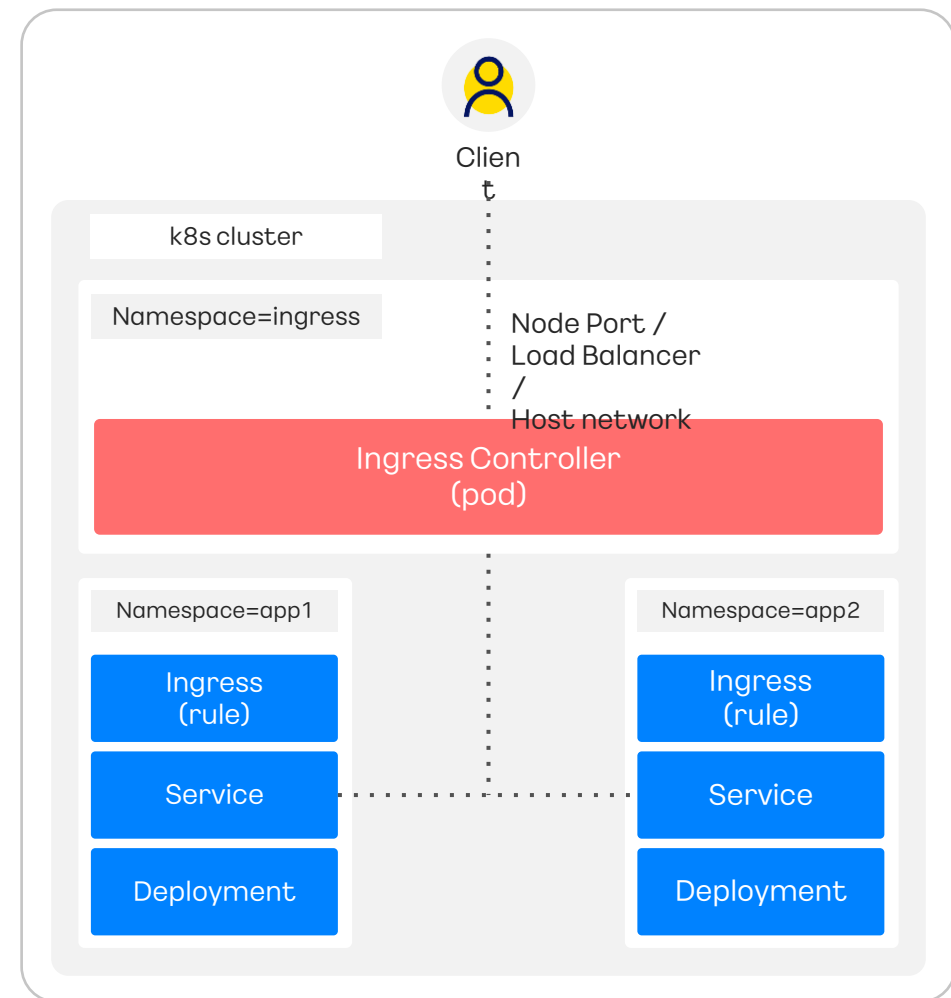
Kubernetes Engine의 사용자 정의 설정 및 웹서버 배포

인그레스 컨트롤러 배포

- Kubernetes Engine 서비스는 인그레스 컨트롤러의 지원을 포함하지 않음
- 따라서, 별도의 인그레스 컨트롤러의 배포 및 운영이 필요

인그레스 컨트롤러란?

- 인그레스(Ingress)에는 클러스터 내부 서비스로 들어오는 네트워크 처리에 대한 규칙들이 정의됨
- 클러스터 외부에서 내부 서비스로 HTTP와 HTTPS 경로를 노출하기 위한 주체로 인그레스 컨트롤러 배포가 필요
- 가이드:
<https://docs.kakaocloud.com/service/container-pack/k8se/how-to-guides/k8se-ingress>



Kubernetes Engine의 사용자 정의 설정 및 웹서버 배포

Load Balancer 생성 및 삭제

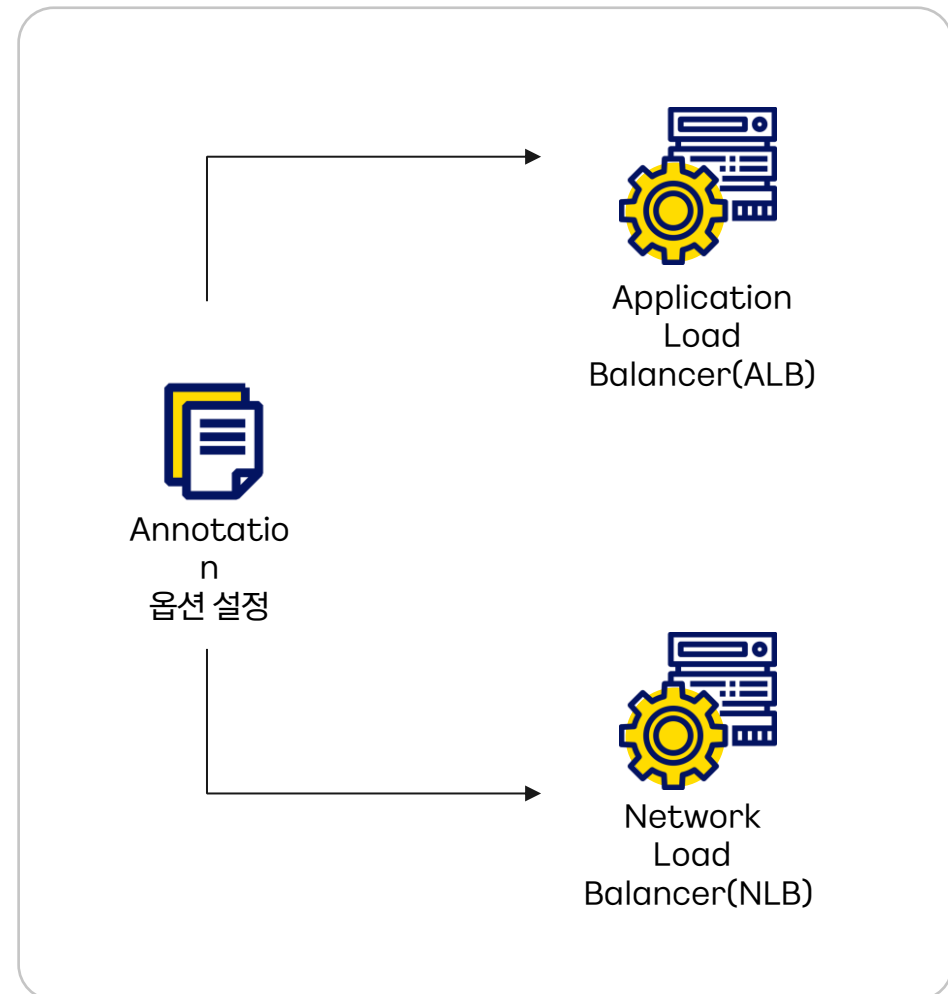
- Kubernetes Engine 서비스에서 로드밸런서 타입의 서비스 생성 시 [Annotation 옵션](#)에 따라, [NLB\(기본 값\)](#) 또는 [ALB](#)가 생성됨.
- 또한, 로드밸런서 생성 시 [Annotation 옵션](#)에 따라, [사설 IP\(기본 값\)](#) 또는 [공인 IP 사용 여부를 설정](#)할 수 있음.
- 로드밸런서 [상세 옵션](#) 설정 가능

Annotation 옵션으로 생성가능한 LB

Application Load Balancer(ALB)

Network Load Balancer(NLB)

* Annotation: key-value를 통해 리소스의 특성을 기록하며 쿠버네티스 시스템에서 필요 정보들을 표시하기 위해 사용



Kubernetes Engine의 사용자 정의 설정 및 웹서버 배포

클러스터에서 영구 볼륨을 사용하기 위한 방법

- 사용자는 일반적으로 스토리지와 Persistent Volume (PV) 객체를 직접 구성해서 영구 볼륨을 사용할 수 있음

주요 용어 및 용도 설명

✓ PV(Persistent Volume)

- PV는 클러스터 수준에서 **영구적인 스토리지를 표현하는 객체**

✓ PVC(Persistent Volume Claim)

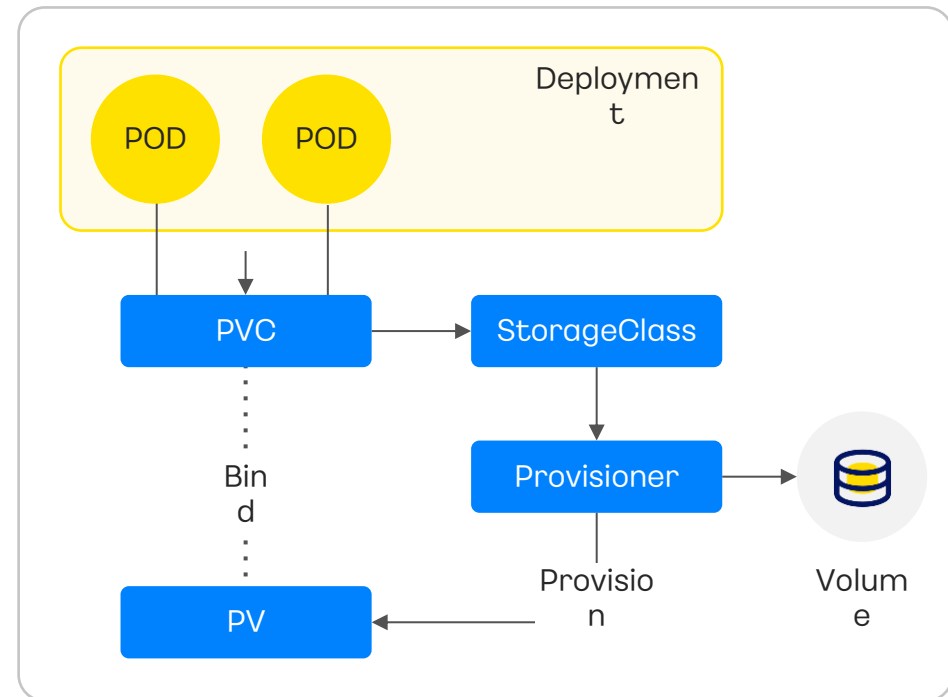
- Pod에서 사용할 **PV를 동적으로 요청하는 객체**로, 실제 PV의 할당을 요청하고 해당 PV를 바인딩 함

✓ StorageClass

- StorageClass는 Provisioner의 타입을 지정하고, **Provisioner가 지원하는 스토리지의 특성을 정의**

✓ Provisioner

- Provisioner는 **스토리지를 동적으로 프로비저닝 하는 컨트롤러** 혹은 드라이버로 StorageClass에 지정된 Provisioner 설정에 맞게 PV를 동적으로 생성



Kubernetes Engine의 사용자 정의 설정 및 웹서버 배포

Kubernetes Engine에서 제공하는 Provisioner

NFS Client Provisioner, CSI Provisioner

- KakaoCloud에서는 두 가지 타입의 Provisioner를 설정해 스토리지와 PV 객체를 쉽게 구성 가능하도록 도움

카카오클라우드와 Provisioner 활용



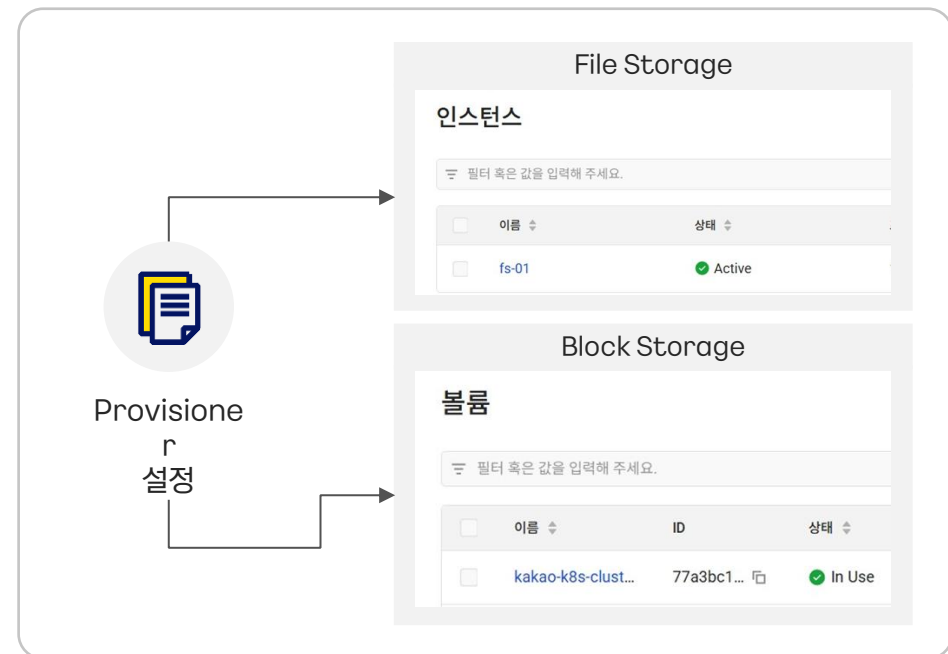
NFS Client Provisioner

- [NFS Client Provisioner](#)를 설정해 카카오클라우드의 File Storage를 영구 볼륨으로 사용 가능



CSI(Container Storage Interface) Provisioner

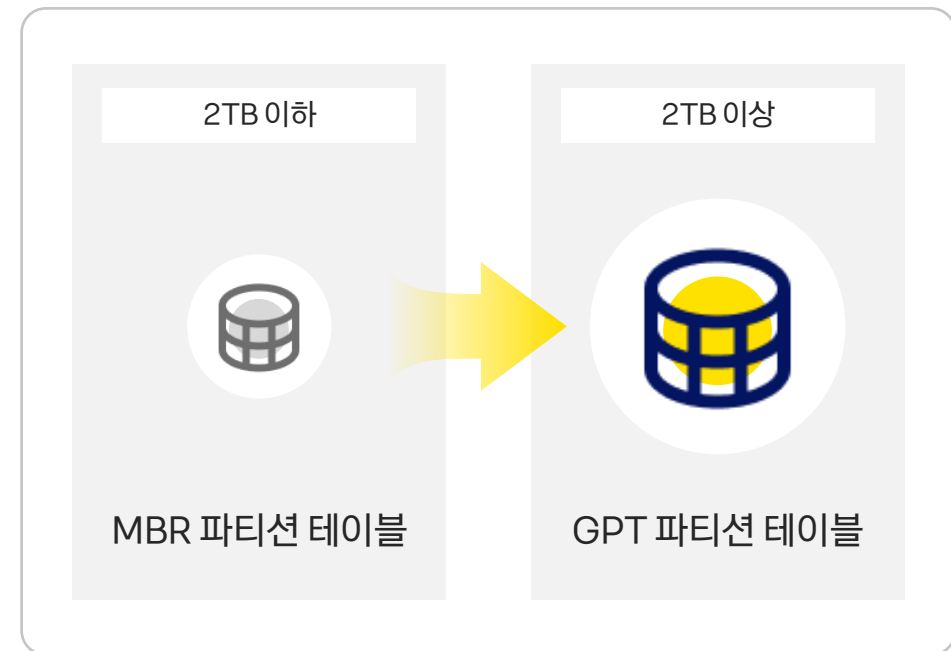
- [CSI Provisioner](#)를 설정해 카카오클라우드의 Block Storage를 영구 볼륨으로 사용가능
-> Bare Metal Server 타입의 노드풀은 지원하지 않음



Kubernetes Engine의 사용자 정의 설정 및 웹서버 배포

루트 볼륨 파티션 테이블 형식 변경

- KakaoCloud에서 제공하는 쿠버네티스 클러스터를 이용 시 각 노드의 볼륨 디스크의 기본 파티션 테이블 형식은 **MBR(Master Boot Record)**로 최대 4개의 파티션, 최대 2TB 용량 이하 디스크로 설정할 수 있음
- 2TB 이상의 노드를 구성해야 하는 경우 **GPT(GUID Partition Table)** 파티션 테이블 형식으로 변경 가능



Lab5 : 인그레스 컨트롤러 배포

(실습 교재 52 page, [Github Link](#))

SECTION.06

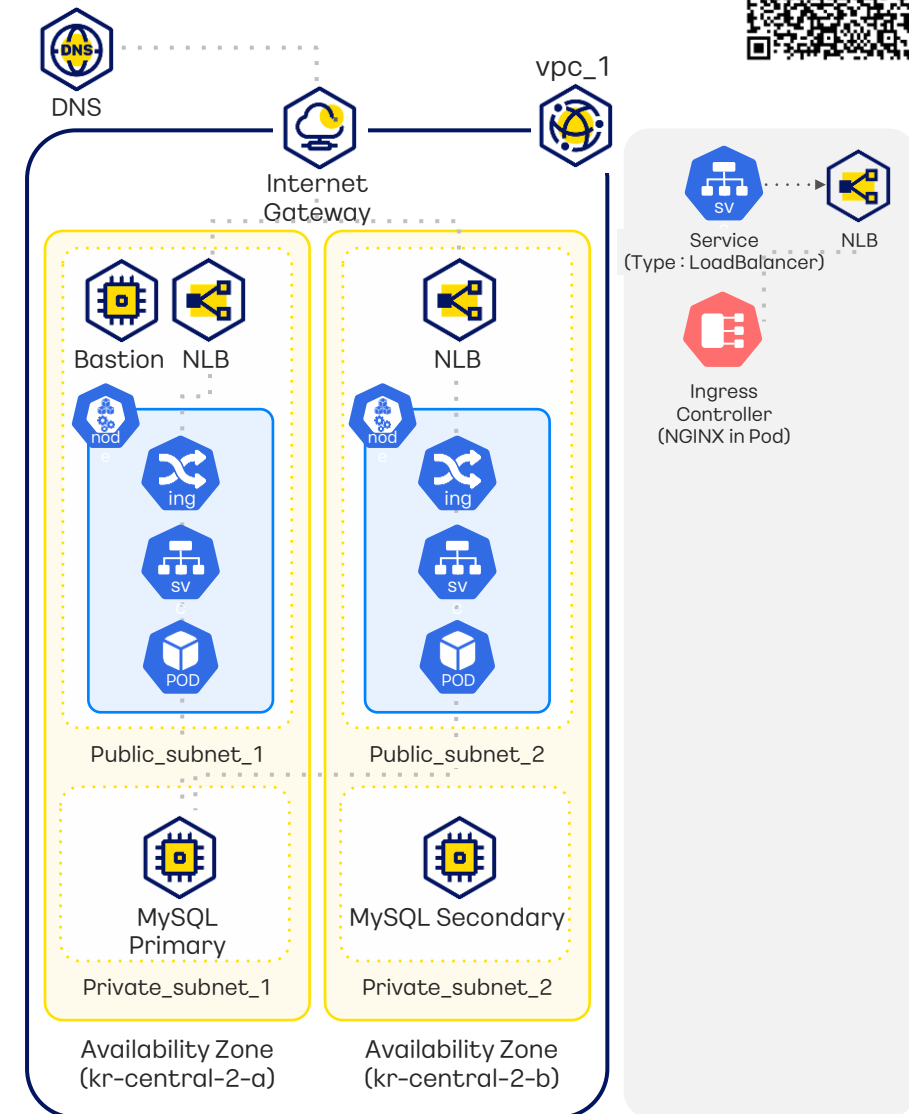


Lab

nginx pod을 위한 deployment, LoadBalancer type의 서비스가 포함된 ingress-nginx controller를 배포하고 ingress-nginx 파드, 서비스, AZ별로 생성된 loadbalancer를 확인하는 실습입니다.

목차

- 1 ingress-nginx controller 배포
- 2 ingress-nginx 파드 및 서비스 상태 확인
- 3 LoadBalancer 확인



Lab6 : Kubernetes Engine 클러스터에 웹서버 자동 배포

SECTION.06



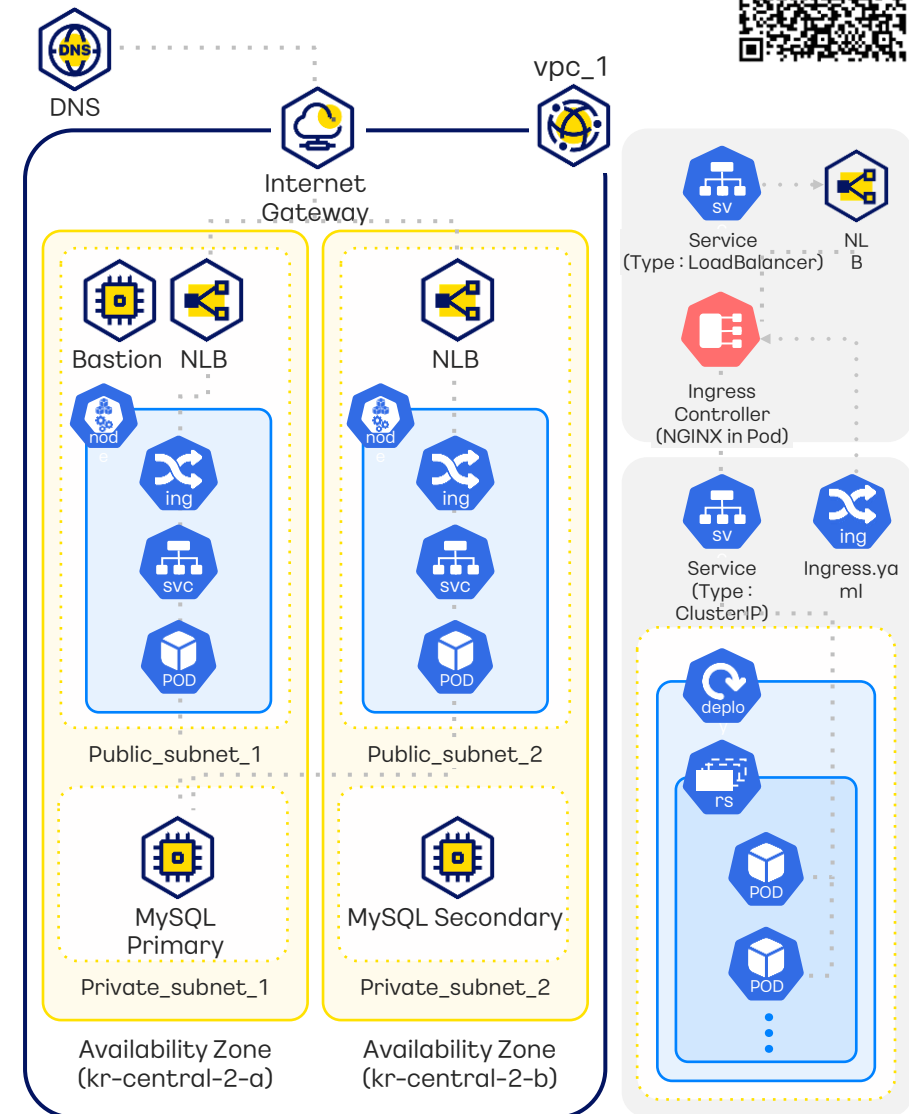
(실습 교재 59 page, [Github Link](#))

Lab

Spring Application 배포를 위해서 다운 받은 YAML 파일을
확인 후 배포하고, 배포된 프로젝트를 웹에서 확인하는 실습입니다.

목차

- 1 YAML 파일 다운 및 설정
- 2 YAML 파일 배포
- 3 배포한 프로젝트 웹에서 확인



Lab7 : Kubernetes Engine 활용하기

SECTION.06



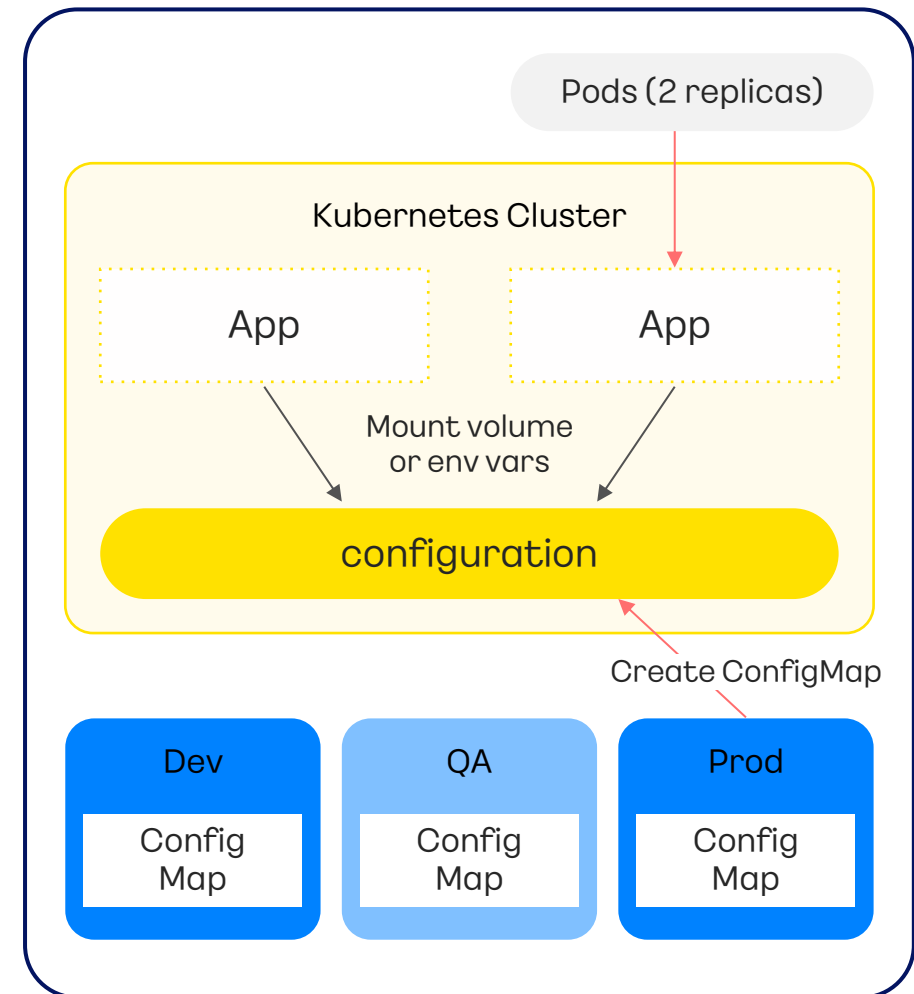
(실습 교재 73 page, [Github Link](#))

Lab

만들어진 웹의 ConfigMap을 변경해서 웹의 구조를 변경하고 확인합니다. 또한 Deployment 리소스의 replicas 값을 변경하여 Pod의 개수를 확인하는 실습 과정을 진행합니다.

목차

- 1 Deployment 리소스의 replicas 값 변경
- 2 YAML 파일을 이용해 배포된 내용 수정해보기
- 3 YAML 파일 배포
- 4 변경된 내용 웹에서 확인



Lab8 : Kubernetes Engine 클러스터에 웹서버 자동화 배포

SECTION.06

(실습 교재 82 page, [Github Link](#))



Lab

지금까지 만든 배포 방식을 자동화하는 도구인 Helm에 대해 실습합니다. Helm을 이용해 Chart를 만들어 배포, 업데이트, 롤백을 진행하는 실습입니다.

목차

- | | | | | | |
|---|--------------------|---|---------------------|---|-----------------------|
| 1 | 기존 리소스 삭제 | 4 | 차트 확인 | 7 | Helm Chart를 이용한 버전 관리 |
| 2 | Helm Chart 설치 | 5 | 차트 문제 검사 | 8 | 롤백 |
| 3 | Helm Chart 프로젝트 설정 | 6 | 차트 설치 시뮬레이션 및 차트 설치 | | |

Lab9 : HPA (Horizontal Pod Autoscaler) 테스트

SECTION.06

(실습 교재 97 page, [Github Link](#))

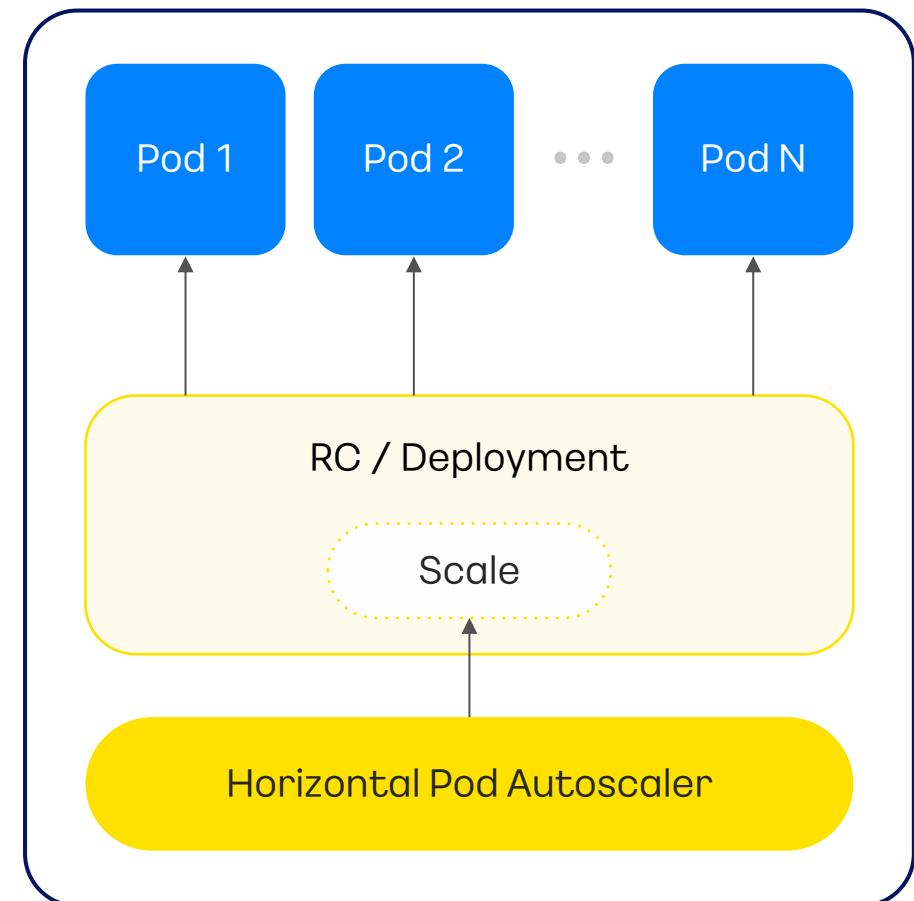


Lab

HPA 옵션을 주어 워크로드 리소스를 자동으로 증가시키는
오토스케일링 실습을 진행합니다.

목차

- 1 HPA 설정
- 2 CPU 부하 발생 및 기능 확인



Lab10 : DNS 서비스를 통해 도메인 주소 연결

SECTION.06

(실습 교재 102 page, [Github Link](#))

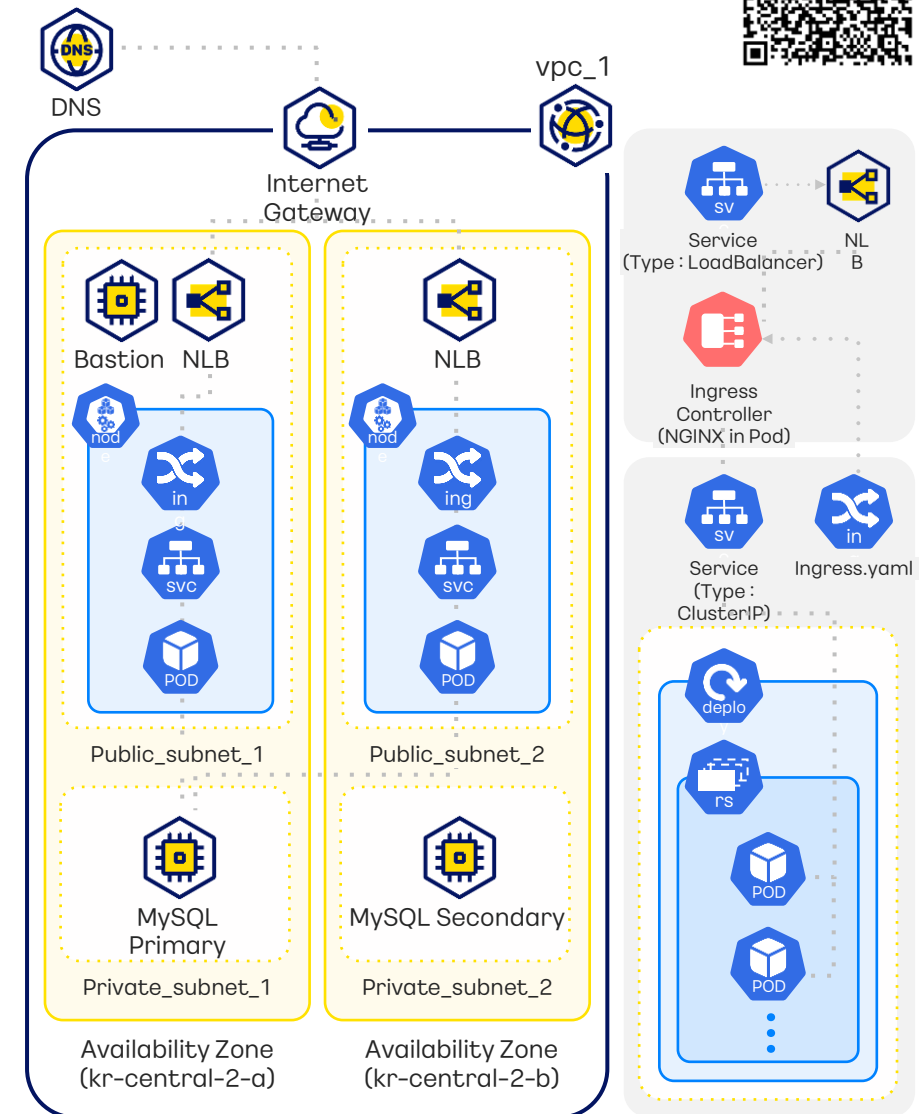


Lab

DNS 서비스를 통해 Public IP로 접속하던 웹서비스를 도메인 주소로 연결합니다. 연결한 도메인을 통해 DNS 서비스가 작동하는지 확인하는 데모를 진행합니다.

목차

- 1 DNS Zone 생성
- 2 네임서버 설정
- 3 서비스 동작 확인





리소스 삭제하기

(삭제 방법 참고: 실습 교재 107 page, [Github Link](#))

불필요한 리소스 삭제는 비용을 절감하고 보안을 강화하며 자원을 최적화하며 확장성을 향상시키고 운영 관리를 간소화하는데 도움을 줍니다.

실습을 진행하면서 생성했던 리소스를 의존성을 고려하여 삭제합니다.

리소스 삭제 순서

아래 순서대로 리소스를 삭제합니다.

k8s 리소스 삭제 → VM → DNS → Container Registry → MySQL → Kubernetes Engine Cluster
→ Public IP → VPC

kakaoenterprise

(주)카카오엔터프라이즈
경기도 성남시 분당구 판교역로 235 에이치스퀘어 N동 7층

7F, 235, Pangyoyeok-ro, Bundang-gu, Seongnam-si,
Gyeonggi-do, 13494, Republic of Korea

KakaoCloud Education Center

<https://edu.kakaocloud.com>



이 문서의 내용과 디자인은 저작권의 보호대상이 되고 부정경쟁방지 및 영업비밀보호에 관한 법률 등 관련 법령에 따라 보호되는 영업비밀 또는 기밀정보를 포함할 수 있습니다. 본 문서에 포함된 정보의 무단 배포, 복사 및 사용은 엄격히 금지됩니다.

The content and design of the document are subject to copyright and may contain trade secrets, privileged and confidential information otherwise protected by applicable law, including the Unfair Competition Prevention and Trade Secret Protection Act. Any unauthorized dissemination, distribution, copying, or use of the information contained in this document is strictly prohibited.

©Kakao Enterprise Corp. All Rights Reserved.

 kakaoenterprise
 kakaoenterprise
 kakaoenterprise

kakaoenterprise